An ICON Package for Experimenting with Euclidean Domains

Author: Lars Warren Ericson
Institution: Courant Institute of Mathematical Sciences, New York University
Report: NYU Computer Science Technical Report #232
Date: August 1986

Abstract

For the purpose of understanding the algebraic algorithms over the Euclidean domain presented in the book of Lipson- [LipsoSla], a small package of routines (about 2000 lines of code) was written in ICON, a software prototyping language developed at the U. of Arizona [Grisw83a,Grisw83b]. This package allows the ICON user to write algorithms which apply to any object of a Euclidean domain, and supplies a paradigm for implementing new Euclidean domains. The package implements those Euclidean domains found in Lipson's book. It turns out that the most difficult part of such a package is the implementation of div and mod for an arbitrary domain. This led the author to exploit a feature of Icon Version 5.10 [Grisw85a] (function call by string image of name) in order to implement a "by-hand" version of the sort of call by inheritance seen in Smalltalk [Ingal78a] and Scratchpad II [Balza84a]. The package may be of use to algebraic algorithm prototypers in the ICON community, or as an adjunct to a course on computer algebra.

1. Introduction

1.1. Programming with Euclidean domains

John Lipson's book. Elements of Algebra and Algebraic Computing, presents a number of interesting symbolic algebraic algorithms, in a style which seems implementable. The only non-trivial implementation detail for the algorithms pfesepted by Lipson is that they assume div and mod operations which are defined on every. Euclidean domain (and, by implication, representations and definitions of these operations for every Euclidean domain). For example, consider his presentation of the FFT algorithm (p. 298);

$$
\begin{array}{c}
\text{procedure } \mathrm{FFT}(N, a(x), \omega, A); \\
\text{if } N = 1 \\
\text{then} \\
\text{Basis. } A_0 := a_0 \\
\text{else} \\
\text{begin} \\
\text{Binary split.} \\
n := N \,/\, 2 \\
b(x) := \sum_{i=0}^{n-1} a_{2i} x^i \\
c(x) := \sum_{i=0}^{n-1} a_{2i+1} x^i \\
\text{Recursive calls.} \\
\mathrm{FFT}(n, b(x), \omega^2, B) \\
\mathrm{FFT}(n, c(x), \omega^2, C) \\
\text{Combine.} \\
\text{for } k := 0 \text{ until } n-1 \text{ do} \\
\text{begin} \\
A_k := B_k + \omega^k \otimes C_k \\
A_{k+n} := B_k - \omega^k \otimes C_k \\
\text{end} \\
\text{end}
\end{array}
$$

The purpose of the package of routines described in this paper is to allow an ICON user to implement an algorithm such as FFT, at about the same level of description as above. By comparison, see Section 3.3.2, which contains our ICON version of the same procedure.

In order to support a high level of description, it must be possible to describe the implementation of particular Euclidean domains, and to describe algorithms which apply generically to all Euclidean domain instances. We do this by deciding which functions are expected of all Euclidean domain implementations (say, div, mod, + and -), and then implementing a "dispatch" version of each of these. The "dispatch" div function inspects the type of its argument (say, integer, polynomial, quotient domain element or modular domain element), and then calls the associated div function in the domain implementation (say divjnteger, dlv_poly, dlv_Q or dlv_mod).

The ability to test the run-time environment is a feature of ICON. Given a string, say "X", and an integer corresponding to a number of formal parameters, say 3, proc("X', 3) will return a procedure (a first-class value in ICON, assignable to variables) if the identifier X is globally to a procedure which is defined to take 3 arguments. Otherwise proc fails. To test for the procedure $\otimes_Z$, we evaluate procC'times" || "*Z", 2), and in general, for some string value X which corresponds to a procedure name, Y a domain name, and i a number of formal parameters, we evaluate proc(X* || "" Y, i), where || is the ICON string concatenation operator. For example, here is the code for the "generic" division operation:

'_' allowed only in math mode

```
$$
\def\odiv{\mathbin{⨸}}
\odiv(a, b) \Leftarrow \Uparrow \text{proc}(\text{"div_"},||,\text{type}(a), 2)(a, b) \ \bl
$$
```

Every implementation of a Euclidean domain must supply certain required procedures. (This notion of "must" corresponds to the idea of a "category" in Scratchpad II.) Optional procedures may be supplied by the domain implementation, but are synthesized if not supplied. The following table lists required, optional and synthesized procedures.

BASIC PROCEDURES FOR COMPUTING WITH DOMAINS

Type	*Required*	*Optional*	*Synthesized*
Constant	0 1		
Operator	abs $\oplus$ $-$ $\otimes$ $\odiv$	mod rem normalize	$\ominus$ exp
Predicates	= < 0 unit $= 0$	$<$	$
Commands		print	pr prs

For a typical domain implementation, which serves as a model for other domain implementations, see for example Section 2.3.1, which describes our implementation of Quotient domains.

A typical application is our implementation of Lipson's algorithm (p. 264) for Newton Interpolation, seen in Section 3.3.3.

1.2 A summary of package facilities for Euclidean domains

The domains supported are as follows:

EUCLIDEAN DOMAIN CONSTRUCTIONS

Primitive domains	*integer*	Machine word integers
	$base_B$	Arbitrary precision unsigned base B integers
	Z	Signed infinite precision integers
Domain constructors	Q	Quotient domain
	modulo	Modular domain
	poly	Polynomial domain
	tpower	Truncated power series domain

The following are the representations in ICON we have adopted for objects in the Euclidean domains we support:

Domain representation

integer	`integer`
$base_B$	`record base_b (base, digits)`
Z	`record Z (sign, mantissa)`
Q	`record Q (dividend, divisor)`
modulo	`record modulo (item, modulus)`
poly	`record poly (terms)`
	`record term (coef, power)`
tpower	`record tpower (poly, N)`

We have implemented the following application algorithms, which may be applied to objects from any Euclidean domain (unless otherwise noted):

Application Algorithms

GCD	greatest common divisor
EUCLID	extended GCD
MOD_RS	polynomial remainder sequence for GCD
PREM	integral domain remainder
E_PRS	polynomial remainder sequence for PREM
INVERSE	inverse of $x \pmod y$

NIA	Newton interpolation algorithm
CRA2, CRA	Chinese remainder algorithm for 2 or more linear congruences
FFT	Fast Fourier Transform
FFI	Fast Fourier Interpolation
NPSI	Newton power series inversion for truncated power series

The system as described is comprised of about 2000 lines of commented ICON code. Supposing that the code defined in the following sections is stored in a file, say euclid, then it may be executed in ICON by adding the statement link euclid to the application program, and then running the ICON translator. The author will gladly supply this code (as is) to any interested user. Mail to ARPA:ericson@nyu or UUCP:{floyd,ihnp4}!cmcl2!csdl!ericson for more information, or via U.S. Mail (with a 600 ft mag tape) to the address listed at the beginning of this report. (The offer last until the author gets sick of making tapes.)

1.3. Our typographical conventions for displaying ICON code

We have dressed up and compressed the syntax of ICON, to give the algorithms presented a more compact, functional appearance.

Icon variables (simple names for single items, and procedure names) may appear as subscripted quantities. This is purely formal, not actual, subscripting. Also, some operator symbols are defined which would not be legal identifiers in ICON (because the characters don't exist in ASCII). Rather than spelling them out, in this report we use the symbol we would have liked to use. The following are some examples of the original code and the fancier notation. Note that underscore ("_") is not a meta-character, but an ordinary character that may appear in identifiers in ICON.

Original ICON	Fancy Notation
`one_base_B`	1_{base_B}
`delta_i_minus_1`	δ_{i-1}
`plus_poly`	$\oplus_{poly}$

For procedure definitions, instead of the obvious

```
procedure F (a, b. c)
  code
end
```

we use the logical-looking

$$\mathrm{F}(a, b, c) \Leftarrow \mathrm{code}\ \blacksquare$$

For `return x` we use $\Uparrow x$, and for `return` we use $\Uparrow$. Instead of `fail` we use $\perp$. All other ICON reserved words are bold-faced.

1.4. Afterthoughts

This code is no longer under development, and seems to be primarily of educational value. In the future, code such as this will be supplanted by far more capable systems such as Scratchpad H, as they become widely available and inexpensive.

As an exercise, the author believes that the code addresses some of the essential software organization issues in computer algebra system design; if the code is to be applied in an educational setting, it would be well to stress to students several other important areas (these areas could form the basis of semester projects to extend the package):

- **Algorithm design.** For example, efficient polynomial greatest common divisor, which has seen many attempts to reduce its complexity. Zippel's notes [Zippe86a] contain a good discussion of this problem.
- **Multivariate polynomials.** This code does not supply any of the several multivariate polynomial representations.
- **Explainability.** Algorithmic (as opposed to "deductive") systems do not explain themselves. An ideal system would supply a proof its conclusion.
- **Numeric-Symbolic Interface.** The results of some computations, for example, polynomial root-finding [Yap86a], are best expressed as numeric approximation intervals, even though they are defining "symbolic" quantities. More work needs to be done to automate the relationship between approximate versus exact computations.

2. Euclidean domains: representation and basic arithmetic

2.1. Generic arithmetic for Euclidean domains

Lipson's book, p. 203, contains a significant proviso:

> We assume that our (Algol-like) language allows for the manipulation of values from an arbitrary Euclidean domain *D* with degree function *d*. In particular we assume that our language provides a *Division Algorithm* in the form of two operations "div" and mod which return, respectively, a preferred quotient and remainder in accordance with the Division Property of a Euclidean domain...

The purpose of this package is to partially implement this proviso. The package implements several primitive domains and *domain constructors*,which are classes of domains composed from other domains.

When a procedure like $\odiv$ or mod is applied to an object which is an instance of a Euclidean domain, the type of the object is determined by inspection. This is either the primitive type, in the case of an instance of a primitive domain, or the type of the "outermost" constructor, in the case of an instance of a composite domain. In the case of required and optional procedures, the run-time environment is then tested to determine whether the domain implementation supplies an operation of this type. If the name of the domain is D, and the procedure name is F, then the run-time environment is tested for a procedure named P_D. For example, $\odiv$ applied to a quotient will look up the procedure $\odiv_Q$. Required procedures must be defined by the domain implementation, otherwise the operation fails. Implementation-optional procedures will synthesize their values if a more domain-specific implementation does not exist.

Constants.

A consequence of the existence of a variety of Euclidean domain instances is that there are a variety of structural representations for 0 and 1. In a given computation, the 0 or 1 used must be of the type of the domain instance. Hence to obtain the correct 0, we evaluate a 0 function which, given an object of the domain instance, returns the 0 of that domain, and similarly for 1.

'_' allowed only in math mode

```
$$
\begin{array}{l}
\mathbf{0}(a) \Leftarrow \Uparrow \text{proc}(\text{"zero_"},|,\text{type}(a), 1)(a) \ \bla
\mathbf{1}(a) \Leftarrow \Uparrow \text{proc}(\text{"one_"},|,\text{type}(a), 1)(a) \ \blac
\end{array}
$$
```

Operators.

The following procedures define the basic arithmetic operations for domains. As noted in Table 1, every domain must supply Abs, $\oplus$, $-$, $\otimes$ and $\odiv$. mod, rem and normalize are optional, and $\ominus$ and exp are synthesized.

$$ \begin{array}{l} \text{Abs}(a) \Leftarrow \Uparrow \text{proc}(\text{"Abs_"},||,\text{type}(a), 1)(a) \ \blacksquare \ \oplus(a, b) \Leftarrow \Uparrow \text{proc}(\text{"plus_"},||,\text{type}(a), 2)(a, b) \ \blacksquare \ \ominus(a, b) \Leftarrow \Uparrow \oplus(a, -(b)) \ \blacksquare \

- (x) \Leftarrow \Uparrow \text{proc}(\text{"minus_"},||,\text{type}(x), 2)(x) \ \blacksquare \ \otimes(a, b) \Leftarrow \Uparrow \text{proc}(\text{"times_"},||,\text{type}(a), 2)(a, b) \ \blacksquare \ \odiv(a, b) \Leftarrow \Uparrow \text{proc}(\text{"div_"},||,\text{type}(a), 2)(a, b) \ \blacksquare \ \text{mod}(a, b) \Leftarrow \ \quad \textbf{if } (x := \text{proc}(\text{"mod_"},||,\text{type}(a), 2)(a, b)) \textbf{ then } \Uparrow x \ \quad \textbf{if } <(b, \mathbf{0}(b)) \textbf{ then } \Uparrow \text{mod}(a, -(b)) \ \quad \Uparrow \text{normalize}(\ \quad\quad \textbf{if } <(a, \mathbf{0}(a)) \ \quad\quad \textbf{then } \oplus(a, \otimes(b,

\oplus(\ominus(-(a), b), \mathbf{1}(a)))) \ \quad\quad \textbf{else } \oplus(a, -(\otimes(b, \odiv(a, b)))) \ \quad) \ \blacksquare \end{array} $$

Example. The polynomials

$$a(x) = x^3 - 2$$
$$b(x) = 2x^2 - 3$$

in the domain of quotients of machine-word integers are denoted within ICON by the record-constructor expressions and variable assignments

$$\text{ax} := \text{poly}([\text{term}(Q(-2,1),0), \text{term}(Q(1,1),3)])$$
$$\text{bx} := \text{poly}([\text{term}(Q(-3,1),0), \text{term}(Q(2,1),2)])$$

pr, a printing control structure, causes expressions to be printed out in a pleasing fashion. The ICON expression `pr{ax, " mod ", bx, " = ", mod(ax, bx)}` will print the following result:

$$(-2)q + 1q \cdot X^3 \bmod (-3)q + 2q \cdot X^2 = (-2)q + \tfrac{3}{2}q \cdot X$$

Similarly, given $c(x) = \frac{3}{2}x - 2$, represented as

$$\text{cx} := \text{poly}([\text{term}(Q(-2,1),0), \text{term}(Q(3,2),1)])$$

The result of evaluating `pr{bx, " mod ", cx, " = ", mod(bx, cx)}` is

$$(-3)q + 2q \cdot X^2 \bmod (-2)q + \tfrac{3}{2}q \cdot X = \tfrac{5}{9}q$$

'_' allowed only in math mode

```
$$
\begin{array}{l}
\text{rem}(a, b) \Leftarrow \\
\quad \Uparrow (\textbf{if } (x := \text{proc}(\text{"rem_"},||,\text{type}(a), 2)(a, b)) \
\quad\quad \textbf{else } \ominus(a, \otimes(\odiv(a, b), b))) \ \blacksquare
\end{array}
$$
```

Example. The polynomials

$$a(x) = 5 - 2x + x^2$$
$$b(x) = 2$$

in the domain of quotients of machine-word integers are denoted with ICON by

'_' allowed only in math mode

```
$$
\begin{array}{l}
\textit{ax} := \text{poly}([\text{term}(\mathcal{Q}(5,1), 0), \text{term}(\mathcal{Q}(-2,1)
\textit{bx} := \text{poly_of}(\mathcal{Q}(2,1))
\end{array}
$$
```

The result of evaluating `pr{ax, " rem ", bx, " = ", rem(ax, bx)}` is

$$5q + (-2)q \cdot X + 1q \cdot X^2 \operatorname{rem} 2q = 0q$$

Similarly, given the equations over the integral domain of polynomials over machine integers denoted by

'_' allowed only in math mode

```
$$
\begin{array}{l}
\textit{ax} := \text{poly}([\text{term}(8, 0), \text{term}(-9, 1), \text{term}(6, 2)]) \\
\textit{bx} := \text{poly_of}(3)
\end{array}
$$
```

The result of evaluating `pr{ax, " rem ", bx, " = ", rem(ax, bx)}` is

$$8 + (-9)X + 6X^2 \operatorname{rem} 3 = 2$$

normalize returns a preferred normal form of a value for a given domain. For example, for quotients, it would be the quotient such that the dividend and divisor have no common non-unit factors. For a modular domain, it would be the least positive element of the equivalence class of the value.

'_' allowed only in math mode

```
$$
\begin{array}{l}
\text{normalize}(a) \Leftarrow \\
\quad \textbf{if } (x := \text{proc}(\text{"normalize_"},|,\text{type}(a), 1)(a)) \textbf{
\quad \Uparrow a \ \blacksquare
\end{array}
$$
```

exp is the Russian Peasants algorithm for exponentiation. Our version Is transliterated from R.B.K. Dewar's SETL implementation of arithmetic for the NYU Ada/Ed system [Dewar81a,Kruch83a].

$$\begin{array}{l} \exp(x,p) \Leftarrow \\ \quad \textbf{if } p = 1 \textbf{ then } \Uparrow x \\ \quad \textbf{else result} := 1(x) \quad u := \text{copy}(x);\ v := p \quad \text{running} := u \quad \textbf{while } v \sim= 0 \textbf{ do} \quad \textbf{if } v2 = 1 \textbf{ then resul} \end{array}$$

Predicates.

All of the predicates defined below except | are required to be defined by a domain instance implementation if they are to be used. However, this is not a minimal set: for example, *is_zero* could be defined in terms of =. | is really not a basic predicate, but since it may be defined in a general way, we include it here.

'_' allowed only in math mode

```
$$
\begin{array}{l}
= (a, b) \Leftarrow \Uparrow \text{proc}(\text{"equal_"},|,\text{type}(a), 2)(a, b) \ \blac
< (a, b) \Leftarrow \Uparrow ((\text{proc}(\text{"less_"},|,\text{type}(a), 2)(a, b)) \math
<0 (x) \Leftarrow \Uparrow \text{proc}(\text{"negative_"},|,\text{type}(x), 1)(x) \ \blacks
\mathit{unit},(x) \Leftarrow \Uparrow \text{proc}(\text{"unit_"},|,\text{type}(x), 1)(x) \
=0 (x) \Leftarrow \Uparrow \text{proc}(\text{"is_zero_"},|,\text{type}(x), 1)(x) \ \blacksq
\end{array}
$$
```

$a \mid c$ (a divides c) if c is a multiple of a, that is, if $\text{rem}(c,a) = 0$.

$$\mid (a,c) \Leftarrow \Uparrow = 0(\text{rem}(c,a)) \ \blacksquare$$

Commands.

Every domain instance D implementation should define a preferred method of printing values in the domain, $print_D$. On top of this, we supply printing control structures *pr* and *prs*. *pr* takes a list of arguments enclosed in braces, and prints them, using the printing procedure appropriate for the type of each argument, followed by a carriage return. *prs* is the same, omitting the carriage return.

prs and *pr* are defined using the user-defined control operation features of ICON 5.10. [Grisw85a, Grisw83a] When *pr* or *prs* is called with a sequence of expressions in braces, the expressions are passed as unactivated co-expressions, which are then activated with the ICON @ operator.

'_' allowed only in math mode

```
$$
\begin{array}{l}
\text{prs},(x) \Leftarrow \text{every } y := \texttt{!x} \textbf{ do } \text{print}(\texttt
\\
\text{pr},(x) \Leftarrow \\
\quad (\text{every } y := \texttt{!x} \textbf{ do } \text{print}(\texttt{@y})) \\
\quad \text{write}() \ \blacksquare \\
\\
\text{print},(x) \Leftarrow \\
\quad \textbf{if } \text{type}(x) \mathrel{==} \text{"list"} \\
\quad \textbf{then } { \text{writes}(\text{"["}) \\
\quad\quad \text{every } y := \texttt{!x}[1:\texttt{*x}] \textbf{ do } { \text{print}(y); \
\quad\quad \text{print}(x[\texttt{*x}]); \text{ writes}(\text{"]"}) } \\
\quad \textbf{else if } pp := \text{proc}(\text{"print_"},||,\text{type}(x), 1) \textbf{ th
\quad \textbf{else if } \text{type}(x) \mathrel{==} \text{"string"} \textbf{ then writes}(x
\quad \textbf{else writes}(\text{image}(x)) \ \blacksquare
\end{array}
$$
```

2.2. Primitive domains

The primitive domains are those which are not constructed from other domains, or which are best thought of as undecomposable. We have three such domains available:

- Arbitrary-precision arbitrary-base integers.
- Arbitrary-precision base 10 integers.
- Ordinary machine integers.

The latter are best unused: ICON does not notify the user of integer multiplication overflow, and overflow can occur very easily in the applications we deal with. For example, subresultant polynomial remainder sequences with cofficients in the 10000 range involve intermediate calculations in the 10000^ range.

2.2.1. Abitrary base, infinite precision non-negative integer

Base B Arithmetic Facilities

Data structures	$base_B$; set_{base}
Constants	0_{base_B}, 1_{base_B}, k_{base_B}
Operators	$\oplus_{base_B}$, $\ominus_{base_B}$, $\otimes_{base_B}$, $\backslash\mathrm{odiv}_{base_B}$, $normalize_{base_B}$
Predicates	$<{*base*{\mathbf{B}}}$, $={*base*{\mathbf{B}}}$
Commands	$print_{base_B}$

Data structures. *base* is a number B such that 1 is less than the maximum machine word integer. Then *digits* is a list of machine word integers less than *base* and greater than 0. Width is the printing width of digits of the base, in terms of decimal digits.

$$\begin{gathered}\text{record } base_B\ (base, digits)\\ \text{global } Base, Width\\ set_{base}(b, w) \Leftarrow\\ Base := b\\ Width := *(b\ \ |\ |\ \ \texttt{""}) - 1\ \blacksquare\end{gathered}$$

Constants.

$$\begin{gathered}0_{base_B}(x) \Leftarrow\Uparrow base_B(x\,.\,base, [0])\ \blacksquare\\ 1_{base_B}(x) \Leftarrow\Uparrow base_B(x\,.\,base, [1])\ \blacksquare\\ k_{base_B}(x) \Leftarrow\Uparrow base_B(\mathrm{Base}, digits_of(abs(x), \mathrm{Base}))\ \blacksquare\\ digits_of(x, B) \Leftarrow \text{if } x < B \text{ then } \Uparrow [x] \text{ else } \Uparrow digits_of(x\ /\ B, B)\ \ |\ |\ |\ \ [mod_{integer}(x, B)]\ \blacksquare\end{gathered}$$

Operators.

The base B addition algorithm is that of Lipson, p. 199. For input it takes a, b, lists of integers $\leq B$, of length m returning $a + b$.

$$\begin{gathered}\oplus_{base_B}(a, b) \Leftarrow\\ B := a\,.\,base\\ \Uparrow base_B(B,\ \oplus_{digits}(a\,.\,digits, b\,.\,digits, B))\ \blacksquare\end{gathered}$$

You can't use 'macro parameter character #' in math mode

```
$$
\begin{array}{l}
\oplus_{digits}(ad, bd, B) \Leftarrow \\
\quad m \mathrel{:=} #ad;\ n \mathrel{:=} #bd \\
\quad \textbf{if } m < n \textbf{ then } { a \mathrel{:=} (list(n - m, 0) \ ||| \ ad);\ b \
\quad \textbf{else if } m > n \textbf{ then } { a \mathrel{:=} ad;\ b \mathrel{:=} list(m -
\quad \textbf{else } { a \mathrel{:=} ad;\ b \mathrel{:=} bd } \\
\quad m \mathrel{:=} #a; \\
\quad c_digits \mathrel{:=} list(m + 1, 0); \\
\quad gamma \mathrel{:=} 0 \\
\quad \textbf{every } i \mathrel{:=} m \textbf{ to } 1 \textbf{ by } -1 \textbf{ do } \\
\quad { t \mathrel{:=} a[i] + b[i] + gamma \\
\quad\quad c_digits[i + 1] \mathrel{:=} mod_{integer}(t, B) \\
\quad\quad gamma \mathrel{:=} t / B } \\
\quad c_digits[1] \mathrel{:=} gamma \\
\\
\quad \Uparrow normalize_{digits}(c_digits) \ \blacksquare
\end{array}
$$
```

Example. The result of evaluating

$$x := base_B(8, [1]);\ y := base_B(8, [7, 7, 7])$$
$$\mathrm{pr} x,\ "+",\ y,\ "=",\ \oplus_{base_B}(x, y)$$

is

1 #8# + 7 7 7 #8# = 1 0 0 0 #8#

The base B subtraction algorithm is Knuth Algorithm 4.3.1 S, transliterated from a SETL implementation of Robert Dewar. Assume $a \geq b$ are lists of integers $\leq B$. Returns $a - b$.

$$\ominus_{base_B}(a, bb) \Leftarrow$$
$$b := copy(bb);\ B := a\,.\,base;\ r$$
$$\text{repeat}$$
$$n := *b\,.\,digits \qquad \text{if } m < n \text{ then pr"ERROR: } base_B \text{ integer subtraction underflow"} \qquad \text{else if } m > n$$

$$\ominus_{digits}\,(a, b, B) \Leftarrow$$
$$u := copy(a)$$
$$v := list(*a - *b, 0) \;\;|\;|\;|\;\; copy(b)$$
$$k := 0$$
$$\text{every } j := *u \text{ to } 1 \text{ by } -1 \text{ do}$$
$$u[j] := u[j] - v[j] + k \qquad \text{if } u[j] < 0 \text{ then } u[j] += B;\; k := -1 \text{ else } k := 0$$
$$\Uparrow normalize_{digits}(u) \;\blacksquare$$

Example.

The result of evaluating

$$x := base_B(10, [1, 0, 0, 5, 6, 3]);\;\; y := base_B(10, [5, 3, 3, 5])$$
$$\mathrm{pr}x,\;\text{" - "},\;y,\;\text{" = "},\;\; \ominus_{base_B}\,(x, y)$$
$$x := base_B(10, [2, 1, 2]);\;\; y := base_B(10, [9, 9])$$
$$\mathrm{pr}x,\;\text{" - "},\;y,\;\text{" = "},\;\; \ominus_{base_B}\,(x, y)$$
$$y := base_B(10, [1, 9, 9])$$
$$\mathrm{pr}x,\;\text{" - "},\;y,\;\text{" = "},\;\; \ominus_{base_B}\,(x, y)$$

is

1 0 0 5 6 3 #10# - 5 3 3 5 #10# = 9 5 2 2 8 #10#
2 1 2 #10# - 9 9 #10# = 1 1 3 #10#
2 1 2 #10# - 1 9 9 #10# = 1 3 #10#

You can't use 'macro parameter character #' in math mode

```
$$
\begin{array}{l}
normalize_{base_{\mathbf{B}}}(r) \Leftarrow \\
\quad d \mathrel{:=} normalize_{digits}(r.digits) \\
\quad \Uparrow base_{\mathbf{B}}(r.base, d) \ \blacksquare \\
\\
normalize_{digits}(d) \Leftarrow \\
\quad \textbf{while } (\#d > 1) \ \& \ (d[1] = 0) \textbf{ do } pop(d) \\
\quad \Uparrow d \ \blacksquare
\end{array}
$$
```

The base B multiplication algorithm is that of Lipson, p. 200. As input it takes a, b, lists of integers $\leq B$, of length m and n. It outputs $a \otimes b$.

$$\otimes_{base_B}\,(a, b) \Leftarrow \Uparrow base_B(a\,.\,base,\; \otimes_{digits}\,(a\,.\,digits, b\,.\,digits, a\,.\,base)) \;\blacksquare$$

You can't use 'macro parameter character #' in math mode

```
$$
\begin{array}{l}
\otimes_{digits}(a, b, B) \Leftarrow \\
\quad m \mathrel{:=} #a \\
\quad n \mathrel{:=} #b \\
\quad c \mathrel{:=} list(m + n, 0) \\
\quad \textbf{every } k \mathrel{:=} 0 \textbf{ to } n - 1 \textbf{ by } 1 \textbf{ do } \\
\quad { \\
\quad\quad gamma \mathrel{:=} 0 \\
\quad\quad \textbf{every } l \mathrel{:=} 0 \textbf{ to } m - 1 \textbf{ by } 1 \textbf{ do
\quad\quad { \\
\quad\quad\quad t \mathrel{:=} a[m - l] * b[n - k] + c[m + n - k - l] + gamma \\
\quad\quad\quad \textbf{if } t < 0 \\
\quad\quad\quad \textbf{then } \text{pr}{\text{"ERROR: Integer overflow in } \otimes_{base_
\quad\quad\quad c[m + n - k - l] \mathrel{:=} mod_{integer}(t, B) \\
\quad\quad\quad gamma \mathrel{:=} t / B \\
\quad\quad } \\
\quad\quad c[n - k] \mathrel{:=} gamma \\
\quad } \\
\\
\quad \Uparrow normalize_{digits}(c) \ \blacksquare
\end{array}
$$
```

Example.

The result of evaluating

$$x := k_{base_B}(28107324);\ y := k_{base_B}(75625)$$
$$\text{pr}x,\ "*",\ y,\ "=",\ \otimes_{base_B}(x, y)$$
$$x := k_{base_B}(28107324);\ y := k_{base_B}(75625)$$
$$\text{pr}x,\ "*",\ y,\ "=",\ \otimes_{base_B}(x, y)$$
$$x := k_{base_B}(7478);\ y := k_{base_B}(4625)$$
$$\text{pr}x,\ "*",\ y,\ "=",\ \otimes_{base_B}(x, y)$$

is

2 8 1 0 7 3 2 4 #10# * 7 5 6 2 5 #10# = 2 1 2 5 6 1 6 3 7 7 5 0 0 #10#
2 8 1 0 7 3 2 4 #10# * 7 5 6 2 5 #10# = 2 1 2 5 6 1 6 3 7 7 5 0 0 #10#
7 4 7 8 #10# * 4 6 2 5 #10# = 3 4 5 8 5 7 5 0 #10#

The following algorithm computes $\frac{a}{b}$ by long division. The design is that of Knuth Algorithm 4.3.1 D [Knuth73a], and the implementation is largely borrowed from a SETL implementation of Robert Dewar [NYU 84a]. Most of the following comments are lifted from the Dewar implementation.

This is by far the most difficult of the four basic operations. This is because the paper and pencil algorithm involves certain amounts of guess work which cannot be programmed directly. The approach (analyzed in detail by Knuth) is to reduce the guess work by computing a rather good guess at each digit of the result, and then correcting if the guess is wrong.

$$\odiv_{base_B}(a,b) \Leftarrow\Uparrow normalize_{base_B}(base_B(a\,.\,base, \odiv_{digits}(a\,.\,digits, b\,.\,digits, a\,.\,base))) \blacksquare$$

$$\begin{array}{c} \odiv_{digits}(a,b,B) \Leftarrow \\ \text{If the divisor is 0, then fail.} \\ \text{if } (*b = 1) \qquad\qquad (b[1] = 0) \text{ ther} \\ \text{If } a \text{ is shorter than } b, \text{ return 0.} \\ \text{if } *a < *b \text{ then } \Uparrow [0] \end{array}$$

The case of a one digit divisor is treated specially. Not only is this more efficient, but the general algorithm assumes that the divisor contains at least two digits. Basically dividing by a single digit is straightforward. Since we can represent numbers up to $B * B - 1$, we can do the steps of the division exactly without any need for guess work. The division is then done left to right.

Missing \begin{array} or extra \end{array}

```
$$
\begin{array}{l}
\textbf{if } *b = 1 \textbf{ then } \\
{ q \mathrel{:=} list(*a, 0) \\
\quad rr \mathrel{:=} 0 \\
\quad \textbf{every } j \mathrel{:=} 1 \textbf{ to } *a \textbf{ do } \\
\quad { du \mathrel{:=} rr * B + a[j] \\
\quad\quad q[j] \mathrel{:=} du / b[1] \\
\quad\quad rr \mathrel{:=} du % b[1] } \\
\quad \Uparrow normalize_{digits}(q) }
\end{array}
$$
```

Otherwise we must commence with the full long division algorithm.

$$\begin{array}{c} u := copy(a) \\ v := copy(b) \\ n := *v \\ m := *u - n \\ q := list(m+1, 0) \end{array}$$

Knuth Step D1. [Normalize] The first step is to multiply both the divisor and dividend by a scale factor. Obviously such scaling does not affect the quotient. The purpose of this scaling is to ensure that the first digit of the divisor is at least $B / 2$. This condition is required for the proper operation of the quotient estimation algorithm used in the division loop. Note that we added an extra digit at the front of the dividend above.

You can't use 'macro parameter character #' in math mode

```
$$
\begin{array}{l}
d \mathrel{:=} B / (v[1] + 1) \\
u \mathrel{:=} \otimes_{digits}(u, [d], B) \\
\textbf{if } #u = m + n \textbf{ then } u \mathrel{:=} [0] \ || \ u \\
v \mathrel{:=} \otimes_{digits}(v, [d], B)
\end{array}
$$
```

Knuth Step D2. [Initialize j] This is the major loop, corresponding to long division steps.

Missing \begin{array} or extra \end{array}

```
$$
\begin{array}{l}
\textbf{every } j \mathrel{:=} 1 \textbf{ to } m + 1 \textbf{ do } \\
{
\end{array}
$$
```

Knuth Step D3. [Calculate q_hat] Guess the next quotient digit by doing a division based on the leading digits. This estimate is never low and at most 2 high.

$$\text{if } u[j] = v[1] \text{ then } qe := B - 1 \text{ else } qe := ((u[j] * B) + u[j+1]) \,/\, v[1]$$

The following loop refines this guess so that it is almost always correct and is at worst one too high (see Knuth [Knuth73a] for proofs).

$$\text{while } (v[2] * qe) > (((u[j] * B) + u[j+1] - (qe * v[1])) * B + u[j+2]) \text{ do } qe - := 1$$

Knuth Step D4. [Multiply and subtract] Now (for the moment accepting the estimate as correct), we subtract the appropriate multiple of the divisor. This is similar to the inner loop of the multiplication routine.

$$
\begin{array}{l}
c := 0 \\
\text{every } k := n \text{ to } 1 \text{ by } -1 \text{ do} \\
du := u[j+k] - (qe * v[k]) + c \quad u[j+k] := du \quad c := du \,/\, B \quad \text{if } u[j+k] < 0 \text{ then } u[j+k] += B;\ c- := 1 \\
u[j] += c
\end{array}
$$

Knuth Step D5,D6. [Test remainder. Add back] If the estimate was one off, then $u[j]$ went negative when the final carry was added above. In this case, we add back the divisor once, and adjust the quotient digit.

Extra close brace or missing open brace

```
$$
\begin{array}{l}
q[j] \mathrel{:=} qe \\
\textbf{if } u[j] < 0 \textbf{ then } \\
{ qe \mathrel{{-}{:=}} 1 \\
\quad c \mathrel{:=} 0 \\
\quad \textbf{every } k \mathrel{:=} n \textbf{ to } 1 \textbf{ by } -1 \textbf{ do } \\
\quad { u[j + k] \mathrel{+{=}} v[k] + c \\
\quad\quad \textbf{if } u[j + k] \geq B \textbf{ then } { u[j + k] \mathrel{{-}{:=}} B;\ c
\quad\quad \textbf{else } c \mathrel{:=} 0 } \\
\quad u[j] \mathrel{+{=}} c } \\
} \\
\Uparrow normalize_{digits}(q) \ \blacksquare
\end{array}
$$
```

Example. The result of evaluating

$$
\begin{array}{c}
\text{every } xy := ![[10,1],[4,2],[27,9],[42,2],[90,1], \\
[188175,325],[188175,579],[188175,580], \\
[188175,578],[121903,5335], \\
[212,99],[115668,75625]] \\
\text{do } x := k_{base_B}(xy[1]);\ y := k_{base_B}(xy[2]) \quad \text{pr} x,\ \text{" / "},\ y,\ \text{" = "},\ \backslash\text{odiv}_{base_B}(x,y)
\end{array}
$$

is

1 0 #10# / 1 #10# = 1 0 #10#
4 #10# / 2 #10# = 2 #10#
2 7 #10# / 9 #10# = 3 #10#
4 2 #10# / 2 #10# = 2 1 #10#
9 0 #10# / 1 #10# = 9 0 #10#
1 8 8 1 7 5 #10# / 3 2 5 #10# = 5 7 9 #10#
1 8 8 1 7 5 #10# / 5 7 9 #10# = 3 2 5 #10#
1 8 8 1 7 5 #10# / 5 8 0 #10# = 3 2 4 #10#
1 8 8 1 7 5 #10# / 5 7 8 #10# = 3 2 5 #10#
1 2 1 9 0 3 #10# / 5 3 3 5 #10# = 2 2 #10#
2 1 2 #10# / 9 9 #10# = 2 #10#
1 1 5 6 6 8 #10# / 7 5 6 2 5 #10# = 1 #10#

Commands. We supply a print command.

> '#' can not be used here

```
$$
\begin{array}{l}
print_{base_{\mathbf{B}}}(b) \Leftarrow \\
\quad \textbf{local } digits \\
\quad writes(b.digits[1],\ \text{" "}) \\
\quad \textbf{every } writes(right(\texttt{!}rest(b.digits),\ Width,\ \text{"0"}),\ \text{"
\quad writes(\text{"#"},\ b.base,\ \text{"#"}) \ \blacksquare
\end{array}
$$
```

Predicates. We supply two predicates, $<*{base*{\mathbf{B}}}$ and $=*{base*{\mathbf{B}}}$.

$$<_{base_B}(a,b) \Leftarrow\Uparrow <_{digits}(a\,.\,digits, b\,.\,digits) \blacksquare$$

$$\begin{array}{c} <_{digits}(a,b) \Leftarrow \\ \text{if } *a < *b \text{ then } \Uparrow \\ \text{else if } (*a > *b) \text{ then } \bot \\ \text{else if } *a = 0 \text{ then } \bot \\ \text{else if } (a[1] > b[1]) \text{ then } \bot \\ \text{else if } (a[1] < b[1]) \text{ then } \Uparrow \\ \text{else } \Uparrow <_{digits}(rest(a), rest(b)) \end{array} \blacksquare$$

$$=_{base_B}(a,b) \Leftarrow\Uparrow =_{digits}(a\,.\,digits, b\,.\,digits) \blacksquare$$

$$\begin{array}{c} =_{digits}(a,b) \Leftarrow \\ \text{if } *a < *b \text{ then } \bot \\ \text{else if } (*a > *b) \text{ then } \bot \\ \text{else if } *a = 0 \text{ then } \Uparrow \\ \text{else if } (a[1] \neq b[1]) \text{ then } \bot \\ \text{else } \Uparrow =_{digits}(rest(a), rest(b)) \end{array} \blacksquare$$

$$rest(x) \Leftarrow \text{if } *x < 2 \text{ then } \Uparrow [] \text{ else } \Uparrow x[2{:}\, *x + 1] \blacksquare$$

2.2.2. Arbitrary precision integer Euclidean domain Z

Integer Arithmetic Facilities

Data structures	Z
Constants	0_Z, 1_Z, k_Z
Operators	$\oplus_Z$, $-_Z$, $\otimes_Z$, $\backslash\text{odiv}_Z$, mod_Z, abs_Z, deg_Z, $normalize_Z$
Predicates	$=_Z$, $<_Z$, $unit_Z$, $>0_Z$, $<0_Z$, $=0_Z$
Commands	$print_Z$

Data structures. *sign* is 1 or -1. *mantissa* is a base $Base$ integer, where the $Base$ is set by k_Z.

$$\text{record } Z\ (sign, mantissa)$$

Constants.

$$0_Z(a) \Leftarrow\Uparrow Z(1, 0_{base_B}(a\,.\,mantissa)) \blacksquare$$
$$1_Z(a) \Leftarrow\Uparrow Z(1, 1_{base_B}(a\,.\,mantissa)) \blacksquare$$

k_Z takes an ICON integer and transforms it into a Z constant.

$$k_Z(x) \Leftarrow$$
$$\text{initial } set_{base}(10000)$$
$$\Uparrow Z(\text{if } x = 0 \text{ then } 1 \text{ else } x \,/\, abs(x),$$
$$base_B(Base, digits_of(abs(x), Base))) \blacksquare$$

Operators.

You can't use 'macro parameter character #' in math mode

```
$$
\begin{array}{l}
\oplus_Z(a, b) \Leftarrow \\
\quad \textbf{if } &lt;0_Z(a) \ &amp; \ &gt;0_Z(b) \textbf{ then } \Uparrow \oplus_Z(b, a)
\quad \Uparrow normalize_Z( \\
\quad\quad \textbf{if } =0_Z(a) \textbf{ then } b \\
\quad\quad \textbf{else if } =0_Z(b) \textbf{ then } a \\
\quad\quad \textbf{else if } (&gt;0_Z(a) \ &amp; \ &gt;0_Z(b)) \ | \ (&lt;0_Z(a) \ &amp; \
\quad\quad \textbf{then } Z(a.sign, \oplus_{base_{\mathbf{B}}}(a.mantissa, b.mantissa)) \\
\quad\quad \textbf{else } { #\ a &gt; 0 \textbf{ and } b &lt; 0,\ \textbf{so...} } \\
\quad\quad\quad \textbf{if } &lt;_{base_{\mathbf{B}}}(a.mantissa, b.mantissa) \\
\quad\quad\quad \textbf{then } Z(-1, \ominus_{base_{\mathbf{B}}}(b.mantissa, a.mantissa)) \
\quad\quad\quad \textbf{else } Z(1, \ominus_{base_{\mathbf{B}}}(a.mantissa, b.mantissa)) }
\quad ) \ \blacksquare
\end{array}
$$
```

Example. The result of evaluating

$$x := k_Z(1);\ y := k_Z(-999)$$
$$\text{pr}x,\ "+",\ y,\ "=",\ \oplus_Z(x, y)$$

is

$1z + (-999z) = (-998z)$

$$-_Z(x) \Leftarrow\Uparrow normalize_Z(Z(-x.sign, x.mantissa)) \blacksquare$$

Example. The result of evaluating

$$x := k_Z(212);\ y := k_Z(-99)$$
$$\text{pr}"-",\ x,\ "=",\ -_Z(x)$$
$$\text{pr}"-",\ y,\ "=",\ -_Z(y)$$

is

$-212z = (-212z)$
$-(-99z) = 99z$

$$\otimes_Z(a,b) \Leftarrow\Uparrow normalize_Z(Z(a\,.\,sign * b\,.\,sign,\ \otimes_{base_B}(a\,.\,mantissa, b\,.\,mantissa))) \blacksquare$$

Example. The result of evaluating

$$\begin{array}{c}\text{every } xy := ![[10,1],[121903,5335],[115668,75625]] \\ \text{do } x := k_Z(xy[1]);\ y := k_Z(xy[2]); \quad \text{pr}x,\ \text{" / "},\ y,\ \text{" = "},\ \backslash\text{odiv}_Z(x,y)\end{array}$$

is

$10z\ /\ 1z = 10z$
$121903z\ /\ 5335z = 22z$
$115668z\ /\ 75625z = 1z$

$$\begin{array}{c} mod_Z(a,b) \Leftarrow \\ \Uparrow (\text{if } <_Z(b, 0_Z(b)) \text{ then } mod_Z(a, -_Z(b)) \\ \text{else if } <_Z(a, 0_Z(a)) \\ \text{then } \oplus_Z(a, -_Z(\otimes_Z(b, \oplus_Z(-_Z(1_Z(a)), \backslash\text{odiv}_Z(a,b))))) \\ \text{else } \oplus_Z(a, -_Z(\otimes_Z(b, \backslash\text{odiv}_Z(a,b))))) \end{array}$$

Example. The result of evaluating

$$\begin{array}{c} x := k_Z(121903);\ y := k_Z(5335) \\ \text{pr}x,\ \text{" mod "},\ y,\ \text{" = "},\ mod_Z(x,y)\end{array}$$

is

$121903z \bmod 5335z = 4533z$

$$abs_Z(x) \Leftarrow\Uparrow Z(1, x\,.\,mantissa) \blacksquare$$

$$deg_Z(x) \Leftarrow\Uparrow x \blacksquare$$
$$normalize_Z(x) \Leftarrow\Uparrow (\text{if } = 0_Z(x) \text{ then } Z(1, x\,.\,mantissa) \text{ else } x) \blacksquare$$

Predicates.

$$ \begin{array}{l} =Z(a, b) \Leftarrow \ \quad \textbf{if } =0_Z(a) \ \& \ =0_Z(b) \textbf{ then } \Uparrow \ \quad \textbf{else if } a.sign \neq b.sign \textbf{ then } \bot \ \quad \textbf{else } \Uparrow = \{base_{\mathbf{B}}\}(a.mantissa, b.mantissa) \ \blacksquare \ \ <Z(a, b) \Leftarrow \ \quad \textbf{if } a.sign < b.sign \textbf{ then } \Uparrow \ \quad \textbf{if } a.sign > b.sign \textbf{ then } \bot \ \quad \textbf{if } a.sign = 1 \textbf{ then } \Uparrow <\{base_{\mathbf{B}}\}(a.mantissa, b.mantissa) \ \quad \textbf{if } a.sign = -1 \textbf{ then } \Uparrow <\{base\{\mathbf{B}\}\}(b.mantissa, a.mantissa) \ \blacksquare \ \ unit_Z(x) \Leftarrow \Uparrow (=Z(x, 1_Z(x)) \ | \ =Z(x, Z(-1, 1\{base\{\mathbf{B}\}\}(x.mantissa)))) \ \blacksquare \ \end{array} $$

> 0_Z(x) \Leftarrow \Uparrow ((x.sign = 1) \ & \ \textbf{not } =0_Z(x)) \ \blacksquare \ <0_Z(x) \Leftarrow \Uparrow ((x.sign = -1) \ & \ \textbf{not } =0_Z(x)) \ \blacksquare \ =0_Z(x) \Leftarrow \Uparrow =*{base*{\mathbf{B}}}(x.mantissa, 0_{base_{\mathbf{B}}}(x.mantissa)) \ \blacksquare \end{array} $$

Commands.

$$
\begin{array}{c}
print_Z(a) \Leftarrow \\
\text{local } digits \\
\text{if } a\,.\,sign < 0 \text{ then } writes(\text{"(-"}) \\
digits := a\,.\,mantissa\,.\,digits \\
\text{every } ch := !digits \text{ do } writes(right(ch,\ Width,\ \text{"0"})) \\
writes(\text{"z"}) \\
\text{if } a\,.\,sign < 0 \text{ then } writes(\text{")"}) \ \blacksquare
\end{array}
$$

2.2.3. Small integers Euclidean domain

We provide the following machine integer arithmetic facilities:

Machine Integer Arithmetic Facilities

Constants	$0_{integer}$, $1_{integer}$
Operators	$\oplus$, $-_{integer}$, $\odot_{integer}$, $\mathit{circleslash}_{integer}$, $rem_{integer}$, $mod_{integer}$, $deg_{integer}$, $abs_{integer}$
Predicates	$= 0_{integer}$, $< 0_{integer}$, $=_{integer}$, $unit_{integer}$
Commands	$print_{integer}$

Constants. We provide constants 0 and 1, as follows:

$$
\begin{array}{c}
0_{integer}(x) \Leftarrow\Uparrow 0 \ \blacksquare \\
1_{integer}(x) \Leftarrow\Uparrow 1 \ \blacksquare
\end{array}
$$

Operators.

$$
\begin{array}{c}
\oplus\,(a, b) \Leftarrow\Uparrow a + b \ \blacksquare \\
-_{i}nteger(x) \Leftarrow\Uparrow -x \ \blacksquare \\
\odot_{integer}\,(a, b) \Leftarrow\Uparrow a * b \ \blacksquare \\
\text{circleslash}_{integer}(a, b) \Leftarrow\Uparrow a \,/\, b \ \blacksquare
\end{array}
$$

$$
\begin{array}{c}
mod_{integer}(a, m) \Leftarrow \\
\text{if } m < 0 \text{ then } m := -m \\
\text{repeat} \\
\text{if } a < 0 \text{ then } a := a + (abs(a \,/\, m) + 1) * m \text{ else } \Uparrow a
\end{array}
$$

rem is not *mod*, because *rem* may be negative, but *mod* is never negative.

$$rem_{integer}(a,b) \Leftarrow\Uparrow a$$

$$deg_{integer}(x) \Leftarrow\Uparrow x \ \blacksquare$$
$$abs_{integer}(x) \Leftarrow\Uparrow abs(x) \ \blacksquare$$

Predicates.

$$=0_{integer}(x) \Leftarrow\Uparrow (x = 0) \ \blacksquare$$
$$<0_{integer}(x) \Leftarrow\Uparrow x < 0 \ \blacksquare$$
$$=_{integer}(a,b) \Leftarrow\Uparrow a = b \ \blacksquare$$
$$unit_{integer}(x) \Leftarrow \text{if } ((x = 1) \mid (x = -1)) \text{ then } \Uparrow x \ \blacksquare$$

Commands.

$$print_{integer}(x) \Leftarrow \text{if } x < 0 \text{ then } writes("(", x, ")") \text{ else } writes(x) \ \blacksquare$$

2.3. Domain constructors

EUCLID provides three classes of domain constructions: quotient domains Q_D, modular domains $D\ /\ (e)$, polynomials $D[x]$ and truncated power series $T(D[[x]])_n$.

2.3.1. Quotient Euclidean domain Q

Quotient Domain Arithmetic Facilities

Data structures	Q
Constants	0_Q, 1_Q, k_{iQ_x}
Operators	$\oplus_Q$, $-{\mathcal{Q}}$, $\otimes{\mathcal{Q}}$, $\odiv_Q$, mod_Q, $normalize_Q$, deg_Q
Predicates	$={\mathcal{Q}}$, $unit{\mathcal{Q}}$
Commands	$print_Q$

Data structures. The domains Q are of the form $Q = \frac{m}{n} \mid m, n \in D, n \neq 0$, for some Euclidean domain D. Elements of such a domain Q are quotients with a dividend and a divisor:

$$\text{record } Q\ (dividend, divisor)$$

Constants.

$$0_Q(x) \Leftarrow\Uparrow Q(0(x\,.\,dividend), 1(x\,.\,dividend)) \blacksquare$$
$$1_Q(x) \Leftarrow\Uparrow Q(1(x\,.\,dividend), 1(x\,.\,dividend)) \blacksquare$$
$$k_{iQ_x}(l, j) \Leftarrow\Uparrow term(Q(l, 1(l)), j) \blacksquare$$

Operators. Let $a = \frac{p}{q}$, $b = \frac{p'}{q}$. Then $a + b = \frac{x}{y}$ where $x = pq' \oplus p'q$, $y = qq'$.

$$\oplus_Q (a, b) \Leftarrow$$
$$\text{local } zz,\ top$$
$$top := \oplus(\otimes(a\,.\,dividend, b\,.\,divisor), \otimes(b\,.\,dividend, a\,.\,divisor))$$
$$zz := 0(a\,.\,dividend)$$
$$\Uparrow \text{if } =_Q (top, zz) \text{ then } Q(zz, 1(a\,.\,dividend))$$
$$\text{else } normalize_Q(Q(top, \otimes(a\,.\,divisor, b\,.\,divisor))) \blacksquare$$

$$-_Q(x) \Leftarrow\Uparrow Q(-(x\,.\,dividend), x\,.\,divisor) \blacksquare$$

$$\otimes_Q (a, b) \Leftarrow\Uparrow normalize_Q(Q(\otimes(a\,.\,dividend, b\,.\,dividend), \otimes(a\,.\,divisor, b\,.\,divisor))) \blacksquare$$

$$\backslash\text{odiv}_Q(a, b) \Leftarrow$$
$$\text{local } zz$$
$$zz := 0(b\,.\,dividend)$$
$$\text{if } =(b\,.\,dividend, zz) \text{ then pr"ERROR: divide by 0 in } Q"$$
$$\text{else } \Uparrow(\text{if } =(a\,.\,divisor, zz) \text{ then } 0_Q(a)$$
$$\text{else } normalize_Q(Q(\otimes(a\,.\,dividend, b\,.\,divisor), \otimes(b\,.\,dividend, a\,.\,divisor)))) \blacksquare$$

There are no remainders in quotient division.

$$mod_Q(a, m) \Leftarrow\Uparrow 0_Q(a) \blacksquare$$

$normalize_Q(x)$ reduces the size of the dividend and divisor, and ensures that any negative sign is in the dividend. Let $g = GCD(x, y)$. Then $normalize_Q(\frac{x}{y}) = \frac{x\backslash\text{odiv}g}{y\backslash\text{odiv}g}$.

$$normalize_Q(x) \Leftarrow$$
$$\text{local } g,\ top,\ bottom$$
$$g := GCD(x\,.\,dividend, x\,.\,divisor)$$
$$top := \backslash\text{odiv}(x\,.\,dividend, g)$$
$$bottom := \backslash\text{odiv}(x\,.\,divisor, g)$$
$$\Uparrow(\text{if } <0(bottom) \text{ then } Q(-(top), -(bottom))$$
$$\text{else } Q(top, bottom)) \blacksquare$$

```
d8g_Q (X) 4= it X ■
```

Predicates.

$\frac{p}{q} = \frac{p'}{q}$ if and only if $pq' = qp'$.

$$=_Q (a,b) \Leftarrow\Uparrow (\ =(\otimes(a\,.\,divisor, b\,.\,dividend), \otimes(b\,.\,divisor, a\,.\,dividend))) \blacksquare$$

Everything is a unit in Q.

$$unit_Q(x) \Leftarrow\Uparrow \blacksquare$$

Commands.

$$\begin{gathered} print_Q(x) \Leftarrow \\ \text{if } =(x\,.\,divisor, 1(x\,.\,divisor)) \\ \text{then } prsx\,.\,dividend,\ \text{"q"} \\ \text{else } prs\text{"("},\ x\,.\,dividend,\ \text{"/"},\ x\,.\,divisor,\ \text{")q"} \blacksquare \end{gathered}$$

2.3.2. Modular Euclidean domain $D\ /\ (x)$

Modular Domain Arithmetic Facilities

Data structures	$modulo$
Constants	0_{modulo}, 1_{modulo}
Operators	$\oplus_{modulo}$, $-{modulo}$, $\otimes$modulo$, $\odiv_{modulo}$, $normalize_{modulo}$, deg_{modulo}
Predicates	$={modulo}$, $unit$modulo$, $< 0_{modulo}$
Commands	$print_{modulo}$

Data structures.

An item from a modular domain, say Z_5, is specified by the item in the "base" domain, plus the modulus.

$$\text{record } modulo\ (item, modulus)$$

Constants.

$$0_{modulo}(a) \Leftarrow\Uparrow modulo(0(a\,.\,item), a\,.\,modulus) \blacksquare$$
$$1_{modulo}(a) \Leftarrow\Uparrow modulo(1(a\,.\,item), a\,.\,modulus) \blacksquare$$

Operators.

$$\oplus_{modulo}\ (a,b) \Leftarrow\Uparrow normalize_{modulo}(modulo(\oplus(a\,.\,item, b\,.\,item), a\,.\,modulus))\ \blacksquare$$
$$-_{modulo}(x) \Leftarrow\Uparrow normalize_{modulo}(modulo(-(x\,.\,item), x\,.\,modulus))\ \blacksquare$$
$$\otimes_{modulo}\ (a,b) \Leftarrow\Uparrow normalize_{modulo}(modulo(\otimes(a\,.\,item, b\,.\,item), a\,.\,modulus))\ \blacksquare$$
$$\backslash odiv_{modulo}(a,b) \Leftarrow\Uparrow normalize_{modulo}(modulo(\otimes(a\,.\,item, INVERSE(b\,.\,item, b\,.\,modulus)), a\,.\,modulus))\ \blacksquare$$

$$normalize_{modulo}(x) \Leftarrow\Uparrow modulo(mod(x\,.\,item, x\,.\,modulus), x\,.\,modulus)\ \blacksquare$$
$$deg_{modulo}(x) \Leftarrow\Uparrow mod(x\,.\,item, x\,.\,modulus)\ \blacksquare$$

Predicates.

$$=_{modulo}\ (a,b) \Leftarrow\Uparrow (\ =(mod(a\,.\,item, a\,.\,modulus), mod(b\,.\,item, b\,.\,modulus)))\ \blacksquare$$
$$unit_{modulo}(a) \Leftarrow\Uparrow (\ =(\ \$\ mod\ \$\ (a\,.\,item, a\,.\,modulus), 1))\ \blacksquare$$

Nothing is negative in a modular domain.

$$<0_{modulo}(a) \Leftarrow \bot\ \blacksquare$$

Commands.

$$print_{modulo}(x) \Leftarrow prs"(",\ x\,.\,item,\ "\ \mathrm{mod}\ ",\ x\,.\,modulus,\ ")"\ \blacksquare$$

2.3.3. Polynomial Euclidean domain $D[x]$

Polynomial Domain Arithmetic Facilities

Data structures	$poly$, $term$; $poly_of$, $0th_coef$, $lead_coef$
Constants	0_{poly}, 1_{poly}, k_{Z_Q}, $k_{Z_{Qx}}$, k_{Z_x}
Operators	$\oplus_{poly}$, $-{poly}$, $\otimes$poly$, $\backslash odiv$_{poly}$, mod_{poly}, $eval_{poly}$, deg_{poly}, $-{deg}$, $\oplus$deg$, $normalize_{poly}$
Predicates	$<{degree}$, $=$poly$, $unit_{poly}$
Commands	$print_{poly}$

Data structures. Polynomials $a(x) \in D[x]$ are finite sums of the form

$$a(x) = \sum_{i=0}^{m} a_i x^i$$

They are represented as lists of terms, in increasing order of power, such that there is always at least one term, 0, if the polynomial is zero. Otherwise the least term may be of any degree.

$$\mathrm{record}\ poly\ (terms)$$
$$poly_of(x) \Leftarrow\Uparrow poly([term(x,0)])\ \blacksquare$$

The coefficient of the constant term as an element of D, if there is a constant term, otherwise 0, may be obtained with:

$$\begin{gathered}0th_{c}oef(fx) \Leftarrow \\ \text{local } a \\ a := fx\,.\,terms[1] \\ \Uparrow (\text{if } a\,.\,power = 0 \text{ then } a\,.\,coef \text{ else } 0(a\,.\,coef)) \blacksquare\end{gathered}$$

The coefficient of the term with the highest degree may be obtained with:

$$lead_{c}oef(ax) \Leftarrow \Uparrow (ax\,.\,terms[*ax\,.\,terms])\,.\,coef \blacksquare$$

A term, say ax^n, is represented as $coef \cdot X^{power}$. It is assumed that coefficient and indeterminate range over the same base domain, and that the power ranges over N.

$$\text{record } term\ (coef, power)$$

Constants.

The zero of the base domain of a coefficient of the polynomial is obtained via:

$$\begin{gathered}0_{poly}(p) \Leftarrow \\ z := 0(p\,.\,terms[1]\,.\,coef) \\ \Uparrow poly([term(z, 0)]) \blacksquare\end{gathered}$$

Example. The result of evaluating

$$\begin{gathered}\text{pr"Q: } 0 = ",\ 0_{poly}(poly([term(Q(-2, 1), 0)])) \\ \text{pr"QZ: } 0 = ",\ 0_{poly}(poly([term(k_{Z_{Qx}}(-2, 0)])))\end{gathered}$$

is

Q: $0 = 0_{q}$
QZ: $0 = 0_{zq}$

The one of the base domain of a coefficient of the polynomial may be obtained with:

$$\begin{gathered}1_{poly}(p) \Leftarrow \\ z := 1(p\,.\,terms[1]\,.\,coef) \\ \Uparrow poly([term(z, 0)]) \blacksquare\end{gathered}$$

An arbitrary-precision rational whole number is obtained with:

$$\begin{gathered}k_{Z_{Q}}(e) \Leftarrow \\ top := k_{Z}(e) \\ \Uparrow Q(top, 1_{Z}(top)) \blacksquare\end{gathered}$$

An arbitrary-precision rational whole number-coefficient indeterminate ex^y is obtained with:

$$k_{Z_{Qx}}(e,y) \Leftarrow\Uparrow term(k_{Z_Q}(e),y) \blacksquare$$

An arbitrary-precision integer-coefficient indeterminate ex^y is obtained with:

$$k_{Z_x}(e,y) \Leftarrow\Uparrow term(k_Z(e),y) \blacksquare$$

Operators.

You can't use 'macro parameter character #' in math mode

```
$$
\begin{array}{l}
\oplus_{poly}(a, b) \Leftarrow \\
\quad \textbf{local } Terms,\ T,\ z \\
\quad Terms \mathrel{:=} \oplus_{terms}(a.terms, b.terms) \\
\quad T \mathrel{:=} [];\ z \mathrel{:=} 0(a.terms[1].coef) \\
\quad \textbf{every } t \mathrel{:=} \texttt{!}Terms \textbf{ do if not } =(t.coef, z) \tex
\quad \Uparrow (\textbf{if } #T > 0 \textbf{ then } poly(T) \textbf{ else } 0(a)) \ \blacks
\end{array}
$$
```

$$\begin{array}{l}
\oplus_{poly}(a, b) \Leftarrow \\
\quad \textbf{local } Terms,\ T,\ z \\
\quad Terms \mathrel{:=} \oplus_{terms}(a.terms, b.terms) \\
\quad T \mathrel{:=} [];\ z \mathrel{:=} 0(a.terms[1].coef) \\
\quad \textbf{every } t \mathrel{:=} \texttt{!}Terms \textbf{ do if not } =(t.coef, z) \tex
\quad \Uparrow (\textbf{if } \#T > 0 \textbf{ then } poly(T) \textbf{ else } 0(a)) \ \blacks
\end{array}$$

$$\text{else} \qquad at := a[1];\ ap := at\,.\,power;\ ac := at\,.\,coef \qquad bt := b[1];\ bp := bt\,.\,power;\ bc := bt\,.\,coef$$

Example. The result of evaluating

$$\begin{array}{c}
ax := poly([term(Q(-2,1),0), term(Q(1,1),3)]) \\
bx := poly([term(Q(-3,1),0), term(Q(2,1),3)]) \\
fx := poly([k_{Z_{Qx}}(-2,0), k_{Z_{Qx}}(1,3)]) \\
gx := poly([k_{Z_{Qx}}(-3,0), k_{Z_{Qx}}(2,3)]) \\
\text{pr"Q: (", } ax, \text{ ") + (", } bx, \text{ ") = ", } \oplus_{poly}(ax, bx) \\
\text{pr"QZ: (", } fx, \text{ ") + (", } gx, \text{ ") = ", } \oplus_{poly}(fx, gx)
\end{array}$$

is

Q: $(-2)q + 1q \cdot X^3) + ((-3)q + 2q \cdot X^3) = (-5)q + 3q \cdot X^3$
QZ: $((-2z)q + 1zq \cdot X^3) + ((-3z)q + 2zq \cdot X^3) = (-5z)q + 3zq \cdot X^3$

$$-_{poly}(x) \Leftarrow$$
$$\text{local } c$$
$$c := []$$
$$\text{every } t := !x\,.\,terms \text{ do } c \mid\mid\mid := [-_{term}(t)]$$
$$\Uparrow poly(c) \blacksquare$$

$$-_{term}(t) \Leftarrow \Uparrow term(-(t\,.\,coef), t\,.\,power) \blacksquare$$

Example. The result of evaluating

$$ax := poly([term(Q(-2,1),0), term(Q(1,1),3)])$$
$$fx := poly([k_{Z_{Qx}}(-2,0), k_{Z_{Qx}}(1,3)])$$
$$\text{pr"Q: - (", } ax, \text{ ") = ", } -_{poly}(ax)$$
$$\text{pr"QZ: - (", } fx, \text{ ") = ", } -_{poly}(fx)$$

is

Q: - $((-2)q + 1q \cdot X^3) = 2q + (-1)q \cdot X^3$
QZ: - $((-2z)q + 1zq \cdot X^3) = 2zq + (-1z)q \cdot X^3$

You can't use 'macro parameter character #' in math mode

```
$$
\begin{array}{l}
\otimes_{poly}(a, b) \Leftarrow \Uparrow \otimes_{poly_terms}(a, b.terms) \ \blacksquare \\
\\
\otimes_{poly_terms}(a, b_terms) \Leftarrow \\
\quad \Uparrow (\textbf{if } #b_terms = 0 \textbf{ then } 0(a) \\
\quad\quad \textbf{else } \oplus_{poly}(\otimes_{poly_term}(a, b_terms[1]), \\
\quad\quad\quad \otimes_{poly_terms}(a, rest(b_terms)))) \ \blacksquare
\end{array}
$$
```

$$\otimes_{poly_term}(a, b_terms) \Leftarrow$$
$$\Uparrow (\text{if } *a\,.\,terms < 2$$
$$\text{then } poly([\otimes_{term_term}(a\,.\,terms[1], b_terms)])$$
$$\text{else } \oplus_{poly}(poly([\otimes_{term_term}(a\,.\,terms[1], b_terms)]),$$
$$\otimes_{poly_terms}(poly(rest(a\,.\,terms)), b_terms))) \blacksquare$$

$$\otimes_{term_term}(a_terms, b_terms) \Leftarrow$$
$$\Uparrow term(\otimes(a_terms\,.\,coef, b_terms\,.\,coef), a_terms\,.\,power + b_terms\,.\,power) \blacksquare$$

Example. The result of evaluating

$$ax := poly([term(Q(-2,1),0), term(Q(1,1),3)])$$
$$bx := poly([term(Q(-3,1),0), term(Q(2,1),3)])$$
$$fx := poly([k_{Z_{Qx}}(-2,0), k_{Z_{Qx}}(1,3)])$$
$$gx := poly([k_{Z_{Qx}}(-3,0), k_{Z_{Qx}}(2,3)])$$
$$\text{pr"Q: (", } ax, \text{ ") * (", } bx, \text{ ") = ", } \otimes_{poly}(ax, bx)$$
$$\text{pr"QZ: (", } fx, \text{ ") * (", } gx, \text{ ") = ", } \otimes_{poly}(fx, gx)$$

is

Q: $((-2)q + 1q \cdot X^3) * ((-3)q + 2q \cdot X^3) = 6q + (-7)q \cdot X^3 + 2q \cdot X^6$
QZ: $((-2z)q + 1zq \cdot X^3) * ((-3z)q + 2zq \cdot X^3) = 6zq + (-7z)q \cdot X^3 + 2zq \cdot X^6$

repeat $m := deg_{poly}(r)$ if $<_{degree}(m,n)$ then $\Uparrow quotient$ else $q := poly([term(\backslash odiv(lead_c o$

Example. The result of evaluating

$$ax := poly_o f(1);\ bx := poly_o f(3)$$
$$\text{pr"integers: ", } ax, \text{ "/", } bx, \text{ " = ", } \backslash odiv_{poly}(ax, bx)$$
$$ax := poly([term(Q(5,9),0)])$$
$$bx := poly([term(Q(-2,1),0), term(Q(3,2),1)])$$
$$fx := poly([term(Q(k_Z(5), k_Z(9)),0)])$$
$$gx := poly([term(Q(k_Z(-2), k_Z(1)),0), term(Q(k_Z(3), k_Z(2)),1)])$$
$$\text{pr"Q: (", } ax, \text{ ") / (", } bx, \text{ ") = ", } \backslash odiv_{poly}(ax, bx)$$
$$\text{pr"QZ: (", } gx, \text{ ") / (", } fx, \text{ ") = ", } \backslash odiv_{poly}(gx, fx)$$
$$ax := poly([term(Q(k_Z(166), k_Z(243)),0), term(Q(k_Z(-275), k_Z(243)),1)])$$
$$bx := poly([term(Q(k_Z(115668), k_Z(75625)),0)])$$
$$\text{pr"QZ[x]: (", } ax, \text{ "/ ", } bx, \text{ ") = ", } \backslash odiv(ax, bx)$$

is

integers: $1 \ / \ 3 = 0$
Q: $((5 \ / \ 9)q) \ / \ ((-2)q + (3 \ / \ 2)q \cdot X) = 0q$
QZ: $((-2z)q + (3z \ / \ 2z)q \cdot X) \ / \ ((5z \ / \ 9z)q) = ((-18z) \ / \ 5z)q + (27z \ / \ 10z)q \cdot X$
$QZ[x]$: $((166z \ / \ 243z)q + ((-275z) \ / \ 243z)q \cdot X) \ / \ ((115668z \ / \ 75625z)q) = (6276875z \ / \ 14053662z)q + ((-$

$$mod_{poly}(a,b) \Leftarrow\Uparrow \ominus(a, \otimes(b, \backslash odiv(a,b))) \ \blacksquare$$

Evaluate $f(x)$ at a, that is evaluate $f(a)$:

$$
\begin{gathered}
eval_{poly}(fx, a) \Leftarrow \\
\text{local } r \\
r := 0(a) \\
\text{every } x := !fx\,.\,terms \text{ do } r := \oplus(r, eval_{term}(x, a)) \\
\Uparrow r \ \blacksquare
\end{gathered}
$$

Evaluate cx^p at $x = a$:

$$eval_{term}(t, a) \Leftarrow\Uparrow \otimes(t\,.\,coef, exp(a, t\,.\,power)) \ \blacksquare$$

Degrees of polynomials are values which may be integers, or the string "$-\infty$". Accordingly, special subtraction and addition procedures are required.

$$
\begin{gathered}
deg_{poly}(x) \Leftarrow \\
\text{if } =_{poly}(x, 0_{poly}(x)) \text{ then } \Uparrow \text{"- infinity"} \\
\text{else } \Uparrow x\,.\,terms[{}^*x\,.\,terms]\,.\,power \ \blacksquare
\end{gathered}
$$

$$
\begin{gathered}
-_{deg}(a, b) \Leftarrow \\
\Uparrow (\text{if } type(a) = \text{"string" then } b \\
\text{else if } type(b) = \text{"string" then } a \\
\text{else } a - b) \ \blacksquare
\end{gathered}
$$

$$
\begin{gathered}
\oplus_{deg}(a, b) \Leftarrow \\
\Uparrow (\text{if } type(a) = \text{"string" then } b \\
\text{else if } type(b) = \text{"string" then } a \\
\text{else } a + b) \ \blacksquare
\end{gathered}
$$

A normal-form polynomial is one whose terms are in normal form (and in ascending order of power).

$$
\begin{gathered}
normalize_{poly}(x) \Leftarrow \\
\text{local } ts \\
ts := [] \\
\text{every } t := !x\,.\,terms \text{ do } ts \ |\,|\,| := [term(normalize(t\,.\,coef), t\,.\,power)] \\
\Uparrow poly(ts) \ \blacksquare
\end{gathered}
$$

Predicates.

You can't use 'macro parameter character #' in math mode

```
$$
\begin{array}{l}
&lt;_{degree}(a, b) \Leftarrow \\
\quad \textbf{if } type(a) = \text{"string"} \\
\quad \textbf{then } \Uparrow not(type(b) = \text{"string"}) \\
\quad \textbf{else } \Uparrow a &lt; b \ \blacksquare \\
\\
=_{poly}(a, b) \Leftarrow \Uparrow =_{terms}(a.terms, b.terms) \ \blacksquare \\
\\
=_{terms}(a, b) \Leftarrow \\
\quad \textbf{if } #a \neq #b \textbf{ then } \bot \\
\quad \textbf{if } #a = 0 \textbf{ then } \Uparrow \\
\quad \textbf{if } =_{term}(a[1], b[1]) \textbf{ then } \Uparrow =_{terms}(rest(a), rest(b)
\\
=_{term}(a, b) \Leftarrow \Uparrow (=(a.coef, b.coef) \ &amp;\ =(a.power, b.power)) \ \blac
\\
unit_{poly}(x) \Leftarrow \Uparrow ((#x.terms = 1) \ &amp;\ (x.terms[1].power = 0) \ &amp;\
\end{array}
$$
```

Commands.

'^' allowed only in math mode

```
$$
\begin{array}{l}
print_{poly}(x) \Leftarrow \\
\quad print_{term}(x.terms[1]) \\
\quad \textbf{every } t \mathrel{:=} \texttt{!}rest(x.terms) \textbf{ do } { writes(\text{"
\\
print_{term}(x) \Leftarrow \\
\quad print(x.coef) \\
\quad \textbf{if } x.power = 1 \textbf{ then } writes(\text{"*X"}) \\
\quad \textbf{else if } x.power > 1 \textbf{ then } prs{\text{"*X^{}"},\ x.power} \ \blacks
\end{array}
$$
```

2.3.4. Truncated Power Series domain $T(D[[x]])_n$

Truncated Power Series Domain Arithmetic Facilities

Data structures	$tpower$
Constants	0_{tpower}, 1_{tpower}
Operators	$\oplus_{tpower}$, $-{tpower}$, $\otimes{tpower}$, $\odiv_{tpower}$, $normalize_{tpower}$
Predicates	$={tpower}$, $unit{tpower}$
Commands	$print_{tpower}$

Data structures.

$$\text{record } tpower\ (Poly, N)$$

Constants.

The zero of the base domain of a coefficient of the polynomial:

$$0_{tpower}(x) \Leftarrow\Uparrow tpower(0_{poly}(x\,.\,Poly), x\,.\,N) \blacksquare$$

The one of the base domain of a coefficient of the polynomial:

$$1_{tpower}(x) \Leftarrow\Uparrow tpower(1_{poly}(x\,.\,Poly), x\,.\,N) \blacksquare$$

Operators.

$$\oplus_{tpower}\ (a, b) \Leftarrow\Uparrow tpower(\ \oplus_{poly}\ (a\,.\,Poly, b\,.\,Poly), a\,.\,N) \blacksquare$$

$$-_{tpower}(x) \Leftarrow\Uparrow tpower(-_{poly}(x\,.\,Poly), x\,.\,N) \blacksquare$$

$$truncate(p, n) \Leftarrow\Uparrow poly(p\,.\,terms[1\colon n+1]) \blacksquare$$

$$\otimes_{tpower}\ (a, b) \Leftarrow\Uparrow tpower(truncate(\ \otimes_{poly}\ (a\,.\,Poly, b\,.\,Poly), a\,.\,N), a\,.\,N) \blacksquare$$

$$\odiv_{tpower}(a, b) \Leftarrow\Uparrow tpower(truncate(\odiv_{poly}(a\,.\,Poly, b\,.\,Poly), a\,.\,N), a\,.\,N) \blacksquare$$

$$normalize_{tpower}(x) \Leftarrow\Uparrow tpower(normalize_{poly}(x\,.\,Poly), x\,.\,N) \blacksquare$$

Predicates.

$$=_{tpower}\ (a, b) \Leftarrow\Uparrow (a\,.\,N = b\,.\,N) \qquad =_{poly}\ (a\,.\,Poly, b\,.\,Poly) \blacksquare$$

$$unit_{tpower}(x) \Leftarrow\Uparrow unit_{poly}(x\,.\,Poly) \blacksquare$$

Commands.

$$print_{tpower}(x) \Leftarrow print_{poly}(x\,.\,Poly) \blacksquare$$

3. Algorithms for various problems over Euclidean domains

We provide algorithms for a number of application areas:

- GCD, linear congruences and Diophantine equations.
- Polynomial remainder sequences.
- Power series and polynomial inversion and interpolation.

In addition, we provide a simple timer facility.

3.1. GCD, Linear Congruences and Diophantine Equations

We provide algorithms for the following applications:

- Euclid's algorithm for greatest common divisor, in simple and extended versions.
- Inverse of $a \bmod m$.
- The Chinese Remainder for 1, 2, or N congruences.
- The solutions to the Diophantine equation $ax + by = c$.

3.1.1. Greatest Common Divisor

We have two versions of Euclid's Algorithm over a Euclidean domain D, from Lipson, p. 226 and p. 209.

GCD(a, b, D)
Input: $a, b \in D$, not both zero.
Output: a gcd of a, b.

$$GCD(a, b) \Leftarrow$$
$$\Uparrow (\text{if } = (b, 0(b)) \text{ then } normalize(a)$$
$$\text{else } GCD(b, mod(a, b))) \blacksquare$$

The following is a table of expressions and their gcd's, as computed via GCD:

Greatest Common Divisors

Domain	A	B	**GCD**
Z	$121903z$	$5335z$	$1z$
Z	$-18z$	$5z$	$1z$
Z	$228z$	$612z$	$12z$
$Q[x]$	$(-2)q + 1q \cdot X^3$	$(-3)q + 2q \cdot X^2$	$(5\ /\ 9)q$
Z_5	$((-2) \bmod 5)$	$((-3) \bmod 5)$	$(2 \bmod 5)$
$Z_5[x]$	$((-2) \bmod 5) + (1 \bmod 5) \cdot X^3$	$((-3) \bmod 5) + (2 \bmod 5) \cdot X^2$	$(3 \bmod 5) + (4$

Domain	A	B	**GCD**
$QZ[x]$	$(166z \ / \ 243z)q + ((-275z) \ / \ 243z)q \cdot X$	$(115668z \ / \ 75625z)q$	$(115668z \ / \ 756$
$QZ[x]$	$(-2z)q + 1zq \cdot X^3$	$(-3z)q + 2zq \cdot X^2$	$(5z \ / \ 9z)q$

EUCLID(a, b)
Input: $a, b \in D$, not both zero.

Unable to render expression.

b

Output: g, s, t such that g is a gcd of a, and

Unable to render expression.

$g = sa + tb$

.

Unable to render expression.

$$
\begin{array}{l}
EUCLID(A, B) \Leftarrow \\
\quad \textbf{local } q,\ a,\ s,\ t \\
\quad a \mathrel{:=} [copy(A), copy(B)] \\
\quad s \mathrel{:=} [1(A), 0(A)] \\
\quad t \mathrel{:=} [0(A), 1(A)] \\
\quad \textbf{while } not(=(a[2], 0(A))) \textbf{ do } \{ \\
\quad\quad q \mathrel{:=} \odiv(a[1], a[2]) \\
\quad\quad a \mathrel{:=} [a[2], \ominus(a[1], \otimes(a[2], q))] \\
\quad\quad s \mathrel{:=} [s[2], \ominus(s[1], \otimes(s[2], q))] \\
\quad\quad t \mathrel{:=} [t[2], \ominus(t[1], \otimes(t[2], q))] \} \\
\quad \Uparrow [normalize(a[1]), normalize(s[1]), normalize(t[1])] \ \blacksquare
\end{array}
$$

The following is a table of expressions and their extended *gcd*'s, as computed via EUCLID:

Extended Greatest Common Divisors

Unable to render expression. **A** , **Unable to render expression.** **B**	**Unable to render expression.** **s** **GCD,** , **Unable to render expression.** **t**
Unable to render expression. 2 , Unable to render expression. 4	Unable to render expression. 2 , Unable to render expression. 1 , Unable to render expression. 0

Unable to render expression. 228 , Unable to render expression. 612	Unable to render expression. 12 , Unable to render expression. (-8) , Unable to render expression. 3
Unable to render expression. 59 , Unable to render expression. 24	Unable to render expression. 1 , Unable to render expression. 11 , Unable to render expression. (-27)

Unable to render expression.
`$(-2)q + 1q \cdot X^3$`
,
Unable to render expression.
`$(-3)q + 2q \cdot X^2$`
	Unable to render expression.
	`$(5/9)q$`
	,
	Unable to render expression.
	`$(-16/9)q + (-4/3)q \cdot X$`
	,
	Unable to render expression.
	`$1q + (8/9)q \cdot X + (2/3)q \cdot X^2$`

Unable to render expression.
`$((-2) \bmod 5) + (1 \bmod 5) \cdot X^3$`
,
Unable to render expression.
`$((-3) \bmod 5) + (2 \bmod 5) \cdot X^2$`
	Unable to render expression.
	`$(3 \bmod 5) + (4 \bmod 5) \cdot X$`
	,
	Unable to render expression.
	`$(1 \bmod 5)$`
	,
	Unable to render expression.
	`$(2 \bmod 5) \cdot X$`

3.1.2. Modular Inverse

Our modular inverse algorithm is that of Lipson, p. 214.

Unable to render expression.

Unable to render expression.

`$(a, m)$`

`$a^{-1} \bmod m$`

INVERSE : Computation of

Unable to render expression.

Unable to render expression.

`$a, m \in D$`

`$D$`

Input: , where is a Euclidean domain.

Unable to render expression.

Unable to render expression.

`$(m, a) = 1$`

`$a^{-1} \bmod m$`

Output: If , then ; otherwise error.

Unable to render expression.

```
$$
\begin{array}{l}
INVERSE(a, m) \Leftarrow \\
\quad \textbf{local } gst \\
\quad gst \mathrel{:=} EUCLID(m, a) \\
\quad \textbf{if } unit(gst[1]) \textbf{ then } \Uparrow mod(\odiv(gst[3], gst[1]), m) \\
\quad \textbf{else } pr{\text{"ERROR: "},\ a,\ \text{"^{}-1 "},\ \text{" mod "},\ m,\ \text
\end{array}
$$
```

A table of modular inverses as computed by INVERSE is as follows:

Unable to render expression. **x**	**modulus**
Unable to render expression. 30	Unable to render expression. 197
Unable to render expression. 16	Unable to render expression. 21
Unable to render expression. 18	Unable to render expression. 21
Unable to render expression. 24	Unable to render expression. 59
Unable to render expression. $(1 \bmod 2) + (1 \bmod 2) \cdot X^2$	Unable to render expression. $(1 \bmod 2) + (1 \bmod 2) \cdot X^2 + (1 \bm

Unable to render expression. **x**	**modulus**
Unable to render expression. $(-3)q + 2q \cdot X^2$	Unable to render expression. $(-2)q + 1q \cdot X^3$

3.1.3. Chinese Remainders and Single-Variable Linear Congruential Systems

We provide three algorithms, **CRA1** for solving equations of the form

Unable to render expression.

$ax \equiv b \pmod m$

, and **CRA2** and **CRA** for solving systems of two or more

Unable to render expression.

$X \equiv a \pmod m$

congruences of the form .

Unable to render expression.

(a, b, m)

CRA1 : Solution of a single linear congruence relation.

Unable to render expression.

Unable to render expression.

a, b, m

$ax \equiv b \pmod m$

Input: such that .

Unable to render expression.

x_1

Output: a particular solution .

Niven and Zuckerman [Niven80a], in their section 2.3 note that, given a congruence

Unable to render expression.

Unable to render expression.

$ax \equiv b \pmod m$

$my \equiv -b \pmod a$

, we can reduce it to . If

Unable to render expression.

y_0

is a solution of the reduced congruence, then

Unable to render expression.

$$x_0 = \frac{my_0 + b}{a}$$

is a solution for the original congruence. They apply the reduction until the congruence is solvable "by inspection". This we do not do. They also have some tricks for size reduction (on p. 43) we will not apply (due to laziness). Our "by inspection" termination condition will be to perform the reduction

Unable to render expression.

Unable to render expression.

```
$a \bmod m = 1$
```

```
$b = 0$
```

until or . Then we return

Unable to render expression.

```
$b \bmod a$
```

, in a recursive setting which builds up the original

Unable to render expression.

```
$x_1$
```

.

Unable to render expression.

```
$$
\begin{array}{l}
CRA1(aa, bb, m) \Leftarrow \\
\quad \textbf{local } a,\ b,\ g \\
\quad g \mathrel{:=} GCD(aa, m) \\
\quad \textbf{if } not\ |(g, bb) \textbf{ then } pr{\text{"ERROR: no solution to linear con
\quad \textbf{else } { a \mathrel{:=} mod(aa, m);\ b \mathrel{:=} mod(bb, m) \\
\quad\quad \textbf{if } =(a, 1(a)) \textbf{ then } \Uparrow b \\
\quad\quad \textbf{else if } =(b, 0(b)) \textbf{ then } \Uparrow 0(b) \\
\quad\quad \textbf{else if } =(a, b) \textbf{ then } \Uparrow 1(b) \\
\quad\quad \textbf{else } \Uparrow \odiv(\oplus(\otimes(m, CRA1(m, -(b), a)), b), a) } \ \b
\end{array}
$$
```

Example. The following results were obtained from executing CRA (the examples are from Niven and Zuckerman [Niven80a], Sect. 2.3):

Unable to render expression.

x

- CRA(7, 1432, 5317): such that

  Unable to render expression.

  $7x \equiv 1432 \bmod 5317$

  is 4762.

  Unable to render expression.

  x

- CRA(863, 880, 2151): such that

  Unable to render expression.

  $863x \equiv 880 \bmod 2151$

  is 173.

  Unable to render expression.

  x

- CRA(589, 509, 817): There is no such that

  Unable to render expression.

  $589x \equiv 509 \bmod 817$

  .

CRA2 and CRA are from Lipson, p. 254 and p. 257.

Unable to render expression.

(r, m, s, n)

CRA2 : Two-congruence Chinese Remainder Algorithm for

Unable to render expression.

Z

$r, m, s, n \in Z$ m

Input: , where ,

Unable to render expression.

n

are relatively prime.

Unable to render expression. Unable to render expression.

$U \in Z$ $U \equiv r \pmod m$

Output: such that and

Unable to render expression.

$U \equiv s \pmod n$

.

Unable to render expression.

```
$$
\begin{array}{l}
CRA2(r, m, s, n) \Leftarrow \\
\quad \textbf{local } c,\ \sigma,\ U \\
\quad c \mathrel{:=} INVERSE(m, n) \\
\quad \sigma \mathrel{:=} mod(\otimes(\ominus(s, r), c), n) \\
\quad U \mathrel{:=} \oplus(r, \otimes(\sigma, m)) \\
\quad \Uparrow U \ \blacksquare
\end{array}
$$
```

Unable to render expression.

Unable to render expression.

x

$x \equiv 6 \pmod 7$

Example. The such that and

Unable to render expression.

$x \equiv 3 \pmod 9$

is 48, as obtained by evaluating CRA2(6, 7, 3, 9).

Unable to render expression. Unable to render expression.

`$(rm_list)$` `$N$`

CRA : -congruence Chinese

Unable to render expression.

`$Z$`

Remainder Algorithm for

Unable to render expression. Unable to render expression.

`$[[r_k, m_k]] \in Z$` `$m_k$`

Input: , where the are relatively prime.

Unable to render expression. Unable to render expression.

`$U \in Z$` `$U \equiv r_i \pmod{m_i}$`

Output: such that .

Unable to render expression.

```
$$
\begin{array}{l}
CRA(rm_list) \Leftarrow \\
\quad \textbf{local } rms,\ rm,\ M,\ U,\ c,\ \sigma \\
\quad rms \mathrel{:=} copy(rm_list) \\
\quad rm \mathrel{:=} pop(rms);\ r \mathrel{:=} rm[1];\ m \mathrel{:=} rm[2] \\
\quad M \mathrel{:=} 1(m) \\
\quad U \mathrel{:=} mod(r, m) \\
\quad \textbf{every } k \mathrel{:=} 1 \textbf{ to } \texttt{*}rms \textbf{ do } { \\
\quad\quad M \mathrel{:=} \otimes(M, m) \\
\quad\quad rm \mathrel{:=} pop(rms);\ r \mathrel{:=} rm[1];\ m \mathrel{:=} rm[2] \\
\quad\quad c \mathrel{:=} INVERSE(M, m) \\
\quad\quad \sigma \mathrel{:=} mod(\otimes(\ominus(mod(U, m), r), c), m) \\
\quad\quad U \mathrel{:=} \oplus(U, \otimes(\sigma, M)) } \\
\quad \Uparrow U \ \blacksquare
\end{array}
$$
```

Unable to render expression.

$u(x)$

Example. The problem is to find in

Unable to render expression.

$Z[x]$

such that

Unable to render expression.

```
$u(x) \bmod 3 = x$
```

,

Unable to render expression.

```
$u(x) \bmod 7 = 1$
```

,

Unable to render expression.

```
$u(x) \bmod 4 = 2x + 3$
```

, and

Unable to render expression.

```
$u(x) \bmod 5 = 3x + 3$
```

.

Unable to render expression.

```
$u(x) = ax + b$
```

Let . Then

Unable to render expression.

```
$$
\begin{array}{ll}
a \bmod 3 = 1 & b \bmod 3 = 0 \\
a \bmod 7 = 0 & b \bmod 7 = 1 \\
a \bmod 4 = 2 & b \bmod 4 = 3 \\
a \bmod 5 = 3 & b \bmod 5 = 3
\end{array}
$$
```

Unable to render expression.

Unable to render expression.

a b

We can solve for and

Unable to render expression.

n

individually using the -congruence CRA algorithm, and we are done. Executing the following code:

Unable to render expression.

```
$$
\begin{array}{l}
a_congruences \mathrel{:=} [[1, 3], [0, 7], [2, 4], [3, 5]] \\
b_congruences \mathrel{:=} [[0, 3], [1, 7], [3, 4], [3, 5]] \\
a \mathrel{:=} CRA(a_congruences) \\
b \mathrel{:=} CRA(b_congruences) \\
ux \mathrel{:=} poly([term(b, 0), term(a, 1)]) \\
\text{pr}{\text{"u(x) = "},\ ux}
\end{array}
$$
```

we discover (final term due to Yap) that

Unable to render expression.

```
$$u(x) = 183 + 238 \cdot X + 3 \cdot 7 \cdot 4 \cdot 5 \sum_{i=0}^{\infty} t_i x^i.$$
```

Unable to render expression.

```
$u$
```

Example. Another example, from Lipson, p. 258, is to compute such that

Unable to render expression.

```
$u \equiv 1 \pmod 3$
```

,

Unable to render expression.

```
$u \equiv 3 \pmod 5$
```

,

Unable to render expression.

```
$u \equiv 0 \pmod 7$
```

,

Unable to render expression.

```
$u \equiv 10 \pmod{11}$
```

.

Executing the following code

Unable to render expression.

```
$$
\text{pr}{CRA([[1, 3], [3, 5], [0, 7], [10, 11]])}
$$
```

Unable to render expression.

```
$U$
```

yields a value of 868 for .

3.1.4. Linear Diophantine Equations in Two Variables

Unable to render expression.

```
$ax + by = c$
```

According to Niven, sect. 5.2, is solvable iff

Unable to render expression. Unable to render expression.

```
$g \mid c$
```

```
$g = \gcd(a, b)$
```

where . If

Unable to render expression.

```
$g \mid c$
```

then all solutions are of the form

Unable to render expression.

```
$$x = x_1 + \frac{b}{g} t, \quad y = y_1 - \frac{a}{g} t$$
```

Unable to render expression.

t

where is an arbitrary integer and

Unable to render expression.

Unable to render expression.

$x = x_1$ $y = y_1$

, is any particular solution of the equation. Particular solutions are obtained by solving one of the linear congruences

Unable to render expression.

$$ax \equiv c \pmod{|b|} \quad \text{or} \quad by \equiv c \pmod{|a|}$$

Unable to render expression.

Unable to render expression.

x_1 y_1

for or , then substituting

Unable to render expression.

Unable to render expression.

y_1 x_1

or into

Unable to render expression.

Unable to render expression.

$ax + by = c$ y_1

to obtain a particular or

Unable to render expression.

x_1

. For computational convenience, if

Unable to render expression.

$|b| \le |a|$

, we solve the first congruence, otherwise we solve the second.

DIOPHANTINE(a, b, c) solves linear Diophantine equations in 2 variables.

Unable to render expression.

Unable to render expression.

Input: a, b, c such that $ax + by = c$.

Unable to render expression.

Unable to render expression.

Output: g, x_1,

Unable to render expression.

y_1

, described above.

Unable to render expression.

```
$$
\begin{array}{l}
DIOPHANTINE(a, b, c) \Leftarrow \\
\quad \textbf{local } gst,\ g,\ x_1,\ y_1 \\
\quad gst \mathrel{:=} EUCLID(a, b) \\
\quad g \mathrel{:=} gst[1];\ t \mathrel{:=} gst[3] \\
\quad \textbf{if } not\ |(g, c) \textbf{ then } pr{\text{"ERROR: Diophantine solution nonex
\quad \textbf{else } { \textbf{if } <(abs(b), abs(a)) \\
\quad\quad \textbf{then } { x_1 \mathrel{:=} CRA1(a, c, abs(b)) \\
\quad\quad\quad y_1 \mathrel{:=} \odiv(\ominus(c, \otimes(a, x_1)), b) } \\
\quad\quad \textbf{else } { y_1 \mathrel{:=} CRA1(b, c, abs(a)) \\
\quad\quad\quad x_1 \mathrel{:=} \odiv(\ominus(c, \otimes(b, y_1)), a) } \\
\quad\quad \Uparrow [g, x_1, y_1] } \ \blacksquare
\end{array}
$$
```

Example. By evaluating DIOPHANTINE(84, 54, -24), we find that all integer solutions

Unable to render expression. Unable to render expression.

(x, y) $84x + 54y = -24$

of the equation are of the form

Unable to render expression. Unable to render expression.

$x = 1 + 9t$ $y = (-2) - 14t$

, .

Example. By evaluating DIOPHANTINE(999, -49, 5000), we find that all integer solutions

Unable to render expression. Unable to render expression.

(x, y) $999x + (-49)y = 5000$

of the equation are of the form

Unable to render expression. Unable to render expression.

$x = 13 + 49t$ $y = 163 - (-999)t$

, .

Example. By evaluating DIOPHANTINE(247, 589, 817), we find that all integer solutions

Unable to render expression. Unable to render expression.

(x, y) $247x + 589y = 817$

of the equation are of the form

Unable to render expression. Unable to render expression.

$x = (-11) + 31t$ $y = 6 - 13t$

, .

3.2 Polynomial remainder sequences

Polynomial remainder sequences are studied as a method of finding variants of the greatest common divisor for elements of integral domains. Variation in the definition is required because integral domains do not support long division. It is also desirable to compute values which share properties of the greatest common divisor (which might then be reclaimed by homomorphic image methods; see Lipson, ch. 8), such that the computation does not suffer the large coefficient growth of Euclid's algorithm on even moderate-sized polynomials. Yap [Yap85a] discusses the issue, presenting an example of Knuth exhibiting the coefficient growth problem. Polynomial remainder sequences are discussed in greater depth in the paper by Loos [Loosa]. We have implemented three variants of polynomial remainder sequence:

- mod-based PRS.
- prem, a pseudo-remainder for division over integral domains, and a prem-based PRS, as defined in Yap [Yap85a].
- Subresultant PRS, as defined in Yap [Yap85a] and based on an algorithm of Collins, as presented by Brown.

3.2.1 MOD-based PRS

The simplest polynomial remainder sequence is simply that of Euclid's algorithm. That is, we define MOD_RS(a, b) to be the PRS of mod(a, b).

Unable to render expression.

```
$$
\begin{array}{l}
MOD_RS(a, b) \Leftarrow \Uparrow [a] \ |||{:=}\ (\textbf{if } =(b, 0(b)) \textbf{ then } [b
\end{array}
$$
```

Unable to render expression.

```
$QZ[x]$
```

Example. In , the remainder sequence of

Unable to render expression.

```
$a(x) = x^5 + 2x^4 + 3x^2 - x + 2$
```

Unable to render expression.

```
$b(x) = 3x^3 - x + 2$
```

as encoded in ICON by

Unable to render expression.

```
$$
\begin{array}{l}
settime() \\
ax \mathrel{:=} poly([k_{Z_{Qx}}(2, 0), k_{Z_{Qx}}(-1, 1), k_{Z_{Qx}}(3, 2), k_{Z_{Qx}}(2,
bx \mathrel{:=} poly([k_{Z_{Qx}}(2, 0), k_{Z_{Qx}}(-1, 1), k_{Z_{Qx}}(3, 3)]) \\
\text{pr}{\text{"QZ[x]: MOD_RS("},\ ax,\ \text{", "},\ bx,\ \text{") = "},\ MOD_RS(ax, bx)}
showtime() \\
\end{array}
$$
```

is

Unable to render expression.

```
$QZ[x]\text{: MOD_RS}(2zq + (-1z)q \cdot X + 3zq \cdot X^2 + 2zq \cdot X^4 + 1zq \cdot X^5,
```

Unable to render expression.

```
$= [2zq + (-1z)q \cdot X + 3zq \cdot X^2 + 2zq \cdot X^4 + 1zq \cdot X^5,\ 2zq + (-1z)q \cd
```

[221033 msecs]

3.2.2 Pseudo-remainder for division over integral domains

Unable to render expression.

px/qx

PREM(px, qx): Pseudo-remainder of in

Unable to render expression. Unable to render expression.

$I[x]$ $I[x]$

, where is an integral domain.

Method:

Unable to render expression.

$d = \deg(p) - \deg(q)$

1. Let

Unable to render expression.

Unable to render expression.

```
$b$
```

```
$q(x)$
```

2. Let = lead coefficient of

Unable to render expression.

```
$\text{rem}(b^{d+1} \cdot px, qx)$
```

3. Return

Unable to render expression.

```
$$
\begin{array}{l}
PREM(px, qx) \Leftarrow \\
\quad \textbf{local } d,\ b \\
\quad d \mathrel{:=} -_{deg}(deg_{poly}(px), deg_{poly}(qx)) \\
\quad b \mathrel{:=} poly_of(lead_coef(qx)) \\
\quad \Uparrow rem(\otimes_{poly}(exp(b, d + 1), px), qx) \ \blacksquare
\end{array}
$$
```

Example. The following table lists values and their pseudo-remainders.

Algorithm! for Tarions problems over Enclidean domains Domain prem(p, q) QZlx] 2DO5427Uz+1785a34z*X StSX2Simi6tST3l82S2MOz -S8S12S9Z79846738 2S2460000000000000Z QZlx] 21z+ (.9z)X+(.4i)'r2+5zX'4+3z*X'6 (-39S35z)+3Q375zTC+15795z*X:2 QZlxl 22q+(-lz)q'X+3zq*X^+22q'X'4+lxq*X'6 2xq+ (-li)q*X+3zq*y3 198zq+(-225z)q*X+306zq'X^ QZlx] 198zq+(-2252)q*X+306zq*X3 iauj+369zq*X iniegen[x]

3.2.3 PREM-based PRS

E_PRS(a, b): Euclidean polynomial remainder sequence.
I.e., a trace of the steps of Euclid's algorithm modified to use PREM.

Unable to render expression.

```
$$
\begin{array}{l}
E_PRS(a, b) \Leftarrow \Uparrow [a] \ |||{:=}\ (\textbf{if } =(b, 0(b)) \textbf{ then } [b]
\end{array}
$$
```

3.2.4 Subresultant PRS

The following algorithm is the Collins-Brown subresultant PRS algorithm, as presented in Yap [Yap85a].

S_PRS: Subresultant polynomial remainder sequence.

Unable to render expression.

```
$p_0, p_1 \in I[x]$
```

Input: polynomials for some integral domain

Unable to render expression.

```
$I$
```

.

Unable to render expression.

```
$(p_0, p_1, \ldots, p_k)$
```

Output: Subresultant PRS such that

Unable to render expression.

```
$p_{k+1} = 0$
```

.

Unable to render expression.

Unable to render expression.

$\delta_i = \deg(p_i) - \deg(p_{i+1})$

$c_i = \text{lead}(p_i)$

Let . Let .

Unable to render expression.

Unable to render expression.

$(R_1, R_2, \ldots, R_k)$

k

Let be a sequence of length defined by

Unable to render expression.

$$R_1 = c_1^{\delta_0}$$

Unable to render expression.

$$R_i = c_i^{\delta_{i-1}} R_{i-1}^{1-\delta_{i-1}}, \quad i = 2, \ldots, k$$

Unable to render expression.

$(\beta_2, \beta_3, \ldots, \beta_k)$

Let be a sequence of length

Unable to render expression.

$k-1$

defined by

Unable to render expression.

$$\beta_2 = (-1)^{\delta_0 + 1}$$

Unable to render expression.

$$\beta_i = (-1)^{1 + \delta_{i-2}} c_{i-2} (R_{i-2})^{\delta_{i-2}}, \quad i = 3, \ldots,

Unable to render expression.

$(p_0, p_1, \ldots, p_k)$

Then we wish to compute the sequence of length

Unable to render expression.

Unable to render expression.

$k+1$

p_0

such that and

Unable to render expression.

p_1

are the given polynomials, and

Unable to render expression.

$$p_i = \frac{\text{PREM}(p_{i-2}, p_{i-1})}{\beta_i}, \quad i = 2, \ldots, k$$

Unable to render expression.

```
$$
\begin{array}{l}
S_PRS(p_0, p_1) \Leftarrow \\
\quad \textbf{local } \delta_0,\ \beta_2,\ p_2,\ x,\ P,\ R_1, \\
\quad\quad \delta_{i-2},\ c_{i-2},\ R_{i-2},\ p_{i-2},\ p_{i-1},\ \beta_i,\ p_i,\ l,\ z \\
\quad \delta_0 \mathrel{:=} \delta_i(p_0, p_1) \\
\quad c_0 \mathrel{:=} c_i(p_0) \\
\quad \beta_2 \mathrel{:=} poly_of(exp(-(1(c_0)), \delta_0 + 1)) \\
\quad p_2 \mathrel{:=} P_i(p_0, p_1, \beta_2);\ z \mathrel{:=} 0(p_2) \\
\quad \textbf{if } =(p_2, z) \textbf{ then } \Uparrow [p_0, p_1] \\
\quad P \mathrel{:=} [p_0, p_1, p_2] \\
\quad R_1 \mathrel{:=} exp(c_i(p_1), \delta_0) \\
\quad \delta_{i-2} \mathrel{:=} \delta_i(p_1, p_2) \\
\quad c_{i-2} \mathrel{:=} c_i(p_1) \\
\quad R_{i-2} \mathrel{:=} R_1 \\
\quad p_{i-2} \mathrel{:=} p_1 \\
\quad p_{i-1} \mathrel{:=} p_2 \\
\quad l \mathrel{:=} 3 \\
\quad \textbf{repeat } \{ \\
\quad\quad \beta_i \mathrel{:=} \beta_i(\delta_{i-2}, c_{i-2}, R_{i-2}) \\
\quad\quad p_i \mathrel{:=} P_i(p_{i-2}, p_{i-1}, \beta_i) \\
\quad\quad \textbf{if } =(p_i, z) \textbf{ then } \Uparrow P \\
\quad\quad \textbf{else } P \ |||{:=}\ [p_i] \\
\quad\quad p_{i-2} \mathrel{:=} p_{i-1} \\
\quad\quad p_{i-1} \mathrel{:=} p_i \\
\quad\quad c_{i-2} \mathrel{:=} c_i(p_{i-2}) \\
\quad\quad R_{i-2} \mathrel{:=} R_i(c_{i-2}, \delta_{i-2}, R_{i-2}) \\
\quad\quad \delta_{i-2} \mathrel{:=} \delta_i(p_{i-2}, p_{i-1}) \} \ \blacksquare \\
\\
\delta_i(p_i, p_{i+1}) \Leftarrow \Uparrow -_{deg}(deg_{poly}(p_i), deg_{poly}(p_{i+1})) \
\\
c_i(p_i) \Leftarrow \Uparrow lead_coef(p_i) \ \blacksquare \\
\\
R_i(c_i, \delta_{i-1}, R_{i-1}) \Leftarrow \\
\quad \Uparrow \otimes(exp(c_i, \delta_{i-1}), exp(R_{i-1}, -_{deg}(\delta_{i-1}, 1))) \ \b
\\
\beta_i(\delta_{i-2}, c_{i-2}, R_{i-2}) \Leftarrow \\
\quad \Uparrow poly_of(\otimes(\otimes(exp(-(1(c_{i-2}))), 1 + \delta_{i-2}), exp(R_{i-2}, \
\\
P_i(p_{i-2}, p_{i-1}, \beta_i) \Leftarrow \Uparrow \odiv(PREM(p_{i-2}, p_{i-1}), \beta_i) \
\end{array}
$$
```

3.3 Power series and polynomial inversion and interpolation

Under this heading we provide the following facilities:

- Newton's method for construction of polynomials by interpolation.
- Fast Fourier Transform (FFT) and Interpolation (FFI).
- Newton's method for truncated power series inversion.

3.3.1 Newton's method for construction of polynomials by interpolation

Unable to render expression.

$F[x]$

NIA(ab_list): Newton's Interpolation Algorithm (CRA for)

Unable to render expression.

Unable to render expression.

$[[a_k, b_k]]$

$U(a_k) = b_k$

Input: such that ,

Unable to render expression.

$U(x) \in F[x]$

Unable to render expression.

$U(x)$

Output:

Unable to render expression.

```
$$
\begin{array}{l}
NIA(ab_list) \Leftarrow \\
\quad \textbf{local } ab_s,\ ab,\ a,\ b,\ Ux,\ Mx,\ c,\ \sigma \\
\quad ab_s \mathrel{:=} copy(ab_list) \\
\quad ab \mathrel{:=} pop(ab_s);\ a \mathrel{:=} ab[1];\ b \mathrel{:=} ab[2] \\
\quad Ux \mathrel{:=} poly_of(b) \\
\quad Mx \mathrel{:=} 1(Ux) \\
\quad \textbf{every } k \mathrel{:=} 1 \textbf{ to } \texttt{*}ab_s \textbf{ do } { \\
\quad\quad Mx \mathrel{:=} \otimes(Mx, \ominus(poly([term(1(b), 1)]), poly_of(a))) \\
\quad\quad ab \mathrel{:=} pop(ab_s);\ a \mathrel{:=} ab[1];\ b \mathrel{:=} ab[2] \\
\quad\quad c \mathrel{:=} \odiv(1(a), eval_{poly}(Mx, a)) \\
\quad\quad \sigma \mathrel{:=} \odiv(\ominus(poly_of(b), poly_of(eval_{poly}(Ux, a))), poly
\quad\quad Ux \mathrel{:=} \oplus(Ux, \otimes(\sigma, Mx)) } \\
\quad \Uparrow Ux \ \blacksquare
\end{array}
$$
```

3.3.2 Fast Fourier Transform (FFT) and Interpolation (FFI)

Unable to render expression.

```
$(N, a(x), \omega, A)$
```

FFT : Fast Fourier Transform

Unable to render expression.

```
$N = 2^m$
```

Input: integer , polynomial

Unable to render expression.

```
$a(x) = \mathrm{sum}(i=0, N-1, a_i \cdot x^i)$
```

, primitive Nth root of unity

Unable to render expression.

```
$\omega$
```

Unable to render expression.

Unable to render expression.

```
$A = (A_0, \ldots, A_{N-1})$
```

```
$A_k = a(\omega^k)$
```

Output: array where

Unable to render expression.

```
$$
\begin{array}{l}
FFT(N, ax, \omega) \Leftarrow \\
\quad \textbf{local } A,\ n,\ bx,\ cx,\ \omega^2,\ B,\ C,\ \omega^k \\
\quad A \mathrel{:=} list(N, []) \\
\quad \textbf{if } N = 1 \ # \text{basis} \\
\quad \textbf{then } A[1] \mathrel{:=} 0th_{coef}(ax) \\
\quad \textbf{else } { n \mathrel{:=} N/2 \ # \text{binary split} \\
\quad\quad bx \mathrel{:=} poly_of_even_powered_terms(ax) \\
\quad\quad cx \mathrel{:=} poly_of_odd_powered_terms(ax) \\
\quad\quad \omega^2 \mathrel{:=} exp(\omega, 2) \\
\quad\quad B \mathrel{:=} FFT(n, bx, \omega^2) \ # \text{recursive calls} \\
\quad\quad C \mathrel{:=} FFT(n, cx, \omega^2) \\
\quad\quad \textbf{every } k \mathrel{:=} 1 \textbf{ to } n \textbf{ do } { \\
\quad\quad\quad \omega^k \mathrel{:=} exp(\omega, k-1) \\
\quad\quad\quad A[k] \mathrel{:=} \oplus(B[k], \otimes(\omega^k, C[k])) \\
\quad\quad\quad A[k+n] \mathrel{:=} \ominus(B[k], \otimes(\omega^k, C[k])) } } \\
\quad \Uparrow A \ \blacksquare
\end{array}
$$
```

Even powered terms.

Unable to render expression.

```
$$
\begin{array}{l}
poly_of_even_powered_terms(ax) \Leftarrow \\
\quad \textbf{local } r \\
\quad r \mathrel{:=} [] \\
\quad \textbf{every } t \mathrel{:=} \texttt{!}ax.terms \\
\quad \textbf{do if } mod_{integer}(t.power, 2) = 0 \textbf{ then } r \ |||{:=}\ [term(t.co
\quad \Uparrow poly(r) \ \blacksquare
\end{array}
$$
```

Odd powered terms.

Unable to render expression.

```
$$
\begin{array}{l}
poly_of_odd_powered_terms(ax) \Leftarrow \\
\quad \textbf{local } r \\
\quad r \mathrel{:=} [] \\
\quad \textbf{every } t \mathrel{:=} \texttt{!}ax.terms \\
\quad \textbf{do if } mod_{integer}(t.power, 2) = 1 \textbf{ then } r \ |||{:=}\ [term(t.co
\quad \textbf{if } \texttt{*}r > 0 \textbf{ then } \Uparrow poly(r) \textbf{ else } \Uparro
\end{array}
$$
```

Unable to render expression.

(N, B, ω)

FFI : Fast Fourier Interpolation

Unable to render expression.

Unable to render expression.

$N = 2^m$ $B = (b_0, \ldots, b_{N-1})$

Input: integer , sample values ,

Unable to render expression.

ω

primitive Nth root of unity

Unable to render expression.

Unable to render expression.

$a(x) = \mathrm{sum}(i=0, N-1, a_i x^i)$ $a(\omega^k) = b_k$

Output: where ,

Unable to render expression.

$k=0..N-1$

Unable to render expression.

```
$$
\begin{array}{l}
FFI(N, B, \omega) \Leftarrow \\
\quad \textbf{local } bx,\ C,\ ax \\
\quad bx \mathrel{:=} polynomialize(B) \\
\quad C \mathrel{:=} FFT(N, bx, \odiv(1(\omega), \omega)) \\
\quad ax \mathrel{:=} polynomialize(\otimes_{vector\ scalar}(C, \odiv(1(N), N))) \\
\quad \Uparrow ax \ \blacksquare
\end{array}
$$
```

Unable to render expression.

```
$$
\begin{array}{l}
polynomialize(B) \Leftarrow \\
\quad \textbf{local } r,\ i \\
\quad r \mathrel{:=} [];\ i \mathrel{:=} 0 \\
\quad \textbf{every } b \mathrel{:=} \texttt{!}B \textbf{ do } \{ \\
\quad\quad \textbf{if } not(=(b, 0(b))) \textbf{ then } r \ |||{:=}\ [term(b, i)] \\
\quad\quad i \mathrel{+{:=}} 1 \} \\
\quad \Uparrow poly(r) \ \blacksquare
\end{array}
$$
```

Unable to render expression.

```
$$
\begin{array}{l}
\otimes_{vector\ scalar}(V, x) \Leftarrow \\
\quad \textbf{local } R,\ i \\
\quad R \mathrel{:=} list(\texttt{*}V);\ i \mathrel{:=} 1 \\
\quad \textbf{every } v \mathrel{:=} \texttt{!}V \textbf{ do } \{ R[i] \mathrel{:=} \otimes(
\quad \Uparrow R \ \blacksquare
\end{array}
$$
```

3.3.3 Newton’s method for truncated power series inversion

NPSI(): Newton's Power Series Inversion Method

Unable to render expression.

$a(t) \bmod t^{2^n} = \mathrm{sum}(i=0, 2^n-1, a_i t^i)$

Input: ,

Unable to render expression.

$a_0 \neq 0$

Unable to render expression.

$x^{(n)}(t) = a(t)^{-1} \bmod t^{2^n}$

Output:

Unable to render expression.

```
$$
\begin{array}{l}
NPSI(at) \Leftarrow \\
\quad \textbf{local } ax,\ xt,\ n \\
\quad ax \mathrel{:=} at.Poly \\
\quad xt \mathrel{:=} poly_of(0th_{coef}(ax)) \\
\quad n \mathrel{:=} log2(\texttt{*}ax.terms) \\
\quad \textbf{every } k \mathrel{:=} 0 \textbf{ to } n-1 \\
\quad \textbf{do } xt \mathrel{:=} \oplus(\oplus(xt, xt), \\
\quad\quad\quad\quad -(\otimes_{poly}(truncate(ax, 2^{k+1}), \otimes(xt, xt)))) \\
\quad \Uparrow tpower(truncate(xt, at.N), at.N) \ \blacksquare
\end{array}
$$
```

Unable to render expression.

```
$$
\begin{array}{l}
log2(x) \Leftarrow \\
\quad \textbf{local } l \\
\quad l \mathrel{:=} 0 \\
\quad \textbf{while } x > 1 \textbf{ do } { x \mathrel{:=} x/2;\ l \mathrel{:=} l + 1 } \\
\quad \Uparrow l \ \blacksquare
\end{array}
$$
```

3.4 A simple timer

A call to `settime()` initializes the timer.

A call to `showtime()` prints the elapsed time since `settime()` was invoked.

Unable to render expression.

```
$$
\begin{array}{l}
\textbf{global } timer \\
showtime() \Leftarrow pr{\text{"["}, &time - timer, \text{" msecs]"}} \ \blacksquare \\
settime() \Leftarrow timer \mathrel{:=} &time \ \blacksquare
\end{array}
$$
```

Appendix. ICON Pretty Printer and Documentation Delaminator

The following documentation filter is inspired by Knuth's *TeX* (specifically the *LaTeX* variant [Lampo83a, Knuth82a].

Blocks of comments are compiled as paragraphs. Paragraphs are demarcated by blank comment lines. Paragraphs are typeset with `.lp`. Code is set off with `.nf`, and `.fi`. We strip any leading white space from comment lines before further processing.

Unable to render expression.

```
$$
\begin{array}{l}
\textbf{global } command_line,\ last_line,\ cur_files,\ read_now,\ words \\
\\
main(x) \Leftarrow \\
\quad \textbf{local } fn \\
\quad words \mathrel{:=} table("") \\
\quad words[\text{"↑"}] \mathrel{:=} \text{"↑"} \\
\quad words[\text{"■"}] \mathrel{:=} \text{"■"} \\
\quad words[\text{"±"}] \mathrel{:=} \text{"±"} \\
\quad command_line \mathrel{:=} x \\
\quad \textbf{if } \texttt{*}command_line > 0 \\
\quad \textbf{then } { fn \mathrel{:=} command_line[1] \\
\quad\quad load_user_keywords(fn \ || \ \text{".keys"}) \\
\quad\quad cur_files \mathrel{:=} [read_now \mathrel{:=} open(fn \ || \ \text{".icn"}, \tex
\quad \textbf{else } cur_files \mathrel{:=} [read_now \mathrel{:=} &input] \\
\quad last_line \mathrel{:=} &null \\
\quad write(\text{".so /usr2/ericson/euclid/lpp/std.me"}) \\
\quad process() \ \blacksquare \\
\\
get_line() \Leftarrow \\
\quad \textbf{local } x \\
\quad x \mathrel{:=} &null \\
\quad \textbf{if } last_line \textbf{ then } { x \mathrel{:=} last_line;\ last_line \mathre
\quad \textbf{else if } x \mathrel{:=} read(read_now) \textbf{ then } \Uparrow x \ \blacksq
\end{array}
$$
```

Reads lines until encountering end of file or `##end` or `##end command`.

Unable to render expression.

```
$$
\begin{array}{l}
process(command) \Leftarrow \\
\quad \textbf{local } line \\
\quad \textbf{while } line \mathrel{:=} get_line() \textbf{ do if } not\ process_line(line,
\\
process_line(line, command) \Leftarrow \\
\quad \textbf{if } line[1:3] \mathrel{==} \text{"##"} \\
\quad \textbf{then } { \textbf{if } line[3:6] \mathrel{==} \text{"■"} \\
\quad\quad \textbf{then } { end_command(command, line[7:\texttt{_}line + 1]);\ \bot } \\
\quad\quad \textbf{else } do_command(line[3:\texttt{_}line + 1]) } \\
\quad \textbf{else if } line[1] \mathrel{==} \text{"#"} \textbf{ then } write_line(line[2:\
\quad \textbf{else } pretty_print(line, command) \\
\Uparrow \ \blacksquare \\
\\
end_command(command, line) \Leftarrow \\
\quad \textbf{if } command \mathrel{\sim}== line \textbf{ then } write(&amp;errout, \text{"
\end{array}
$$
```

If command is non-null then `##■end` command should match command.

For interpreting `##` commands

Unable to render expression.

```
$$
\begin{array}{l}
do_command(line) \Leftarrow \\
\quad \textbf{local } command,\ args \\
\quad x \mathrel{:=} (upto(\mathord{\sim}&amp;lcase, line) \ | \ (\texttt{_}line + 1)) \\
\quad command \mathrel{:=} line[1:x] \\
\quad args \mathrel{:=} line[x + 1:\texttt{_}line + 1] \\
\quad \textbf{if } not(y \mathrel{:=} proc(\text{"do_"} \ || \ command, 2)) \\
\quad \textbf{then } write(&amp;errout, \text{"ERROR: Unknown command: "}, command) \\
\quad \textbf{else } y(args) \ \blacksquare
\end{array}
$$
```

`##list` and `##end list`.

Unable to render expression.

```
$$
\begin{array}{l}
do_list(args) \Leftarrow \\
\quad \textbf{local } line \\
\quad write(\text{".(l I F"}) \\
\quad \textbf{while } line \mathrel{:=} get_line() \\
\quad \textbf{do if } line[1:3] \mathrel{==} \text{"##"} \\
\quad\quad \textbf{then } { \textbf{if } line[3:6] \mathrel{==} \text{"■"} \\
\quad\quad\quad \textbf{then } { write(\text{".)l"});\ end_command(command, line[7:\texttt{
\quad\quad\quad \textbf{else } do_command(line[3:\texttt{_}line + 1]) } \\
\quad\quad \textbf{else if } line[1] \mathrel{==} \text{"#"} \\
\quad\quad \textbf{then } { line \mathrel{:=} line[2:\texttt{_}line + 1] \\
\quad\quad\quad\quad \textbf{repeat if } upto(\text{' '}, line[1]) \\
\quad\quad\quad\quad \textbf{then } line \mathrel{:=} line[2:\texttt{_}line + 1] \textbf{ e
\quad\quad\quad\quad \textbf{if } \texttt{*}line &gt; 0 \textbf{ then } write(\text{"● "},
\quad\quad \textbf{else } pretty_print(line, command) \\
\quad write(\text{".)l"}) \ \blacksquare
\end{array}
$$
```

`##section <I> <title>` and `##end section <I>`.

Section nestings are relative to the file, from 1 on up. An `##include` file's nestings are relative to the current level of the including file plus previous cumulative nesting. I.e., if cumulative nesting is 3, and nesting in the including file is 2, then 1 in the included file translates to 6 in the final output.

Unable to render expression.

```
$$
\begin{array}{l}
do_section(args) \Leftarrow \\
\quad x \mathrel{:=} (upto(\mathord{\sim}(\text{'0123456789'}), args) \ | \ (\texttt{_}args
\quad level \mathrel{:=} args[1:x] + 0 \\
\quad title \mathrel{:=} args[x + 1:\texttt{_}args + 1] \\
\quad write(\text{".sh "}, level, \text{" "}, title) \\
\quad write(\text{".sp 2v0lp"}) \\
\quad process(\text{"section "} \ || \ level) \ \blacksquare
\end{array}
$$
```

`##skip` and `##end skip`.

Deletes *everything* between skip and end skip.

Unable to render expression.

```
$$
\begin{array}{l}
do_skip(x) \Leftarrow \\
\quad \textbf{local } line \\
\quad \textbf{while } line \mathrel{:=} get_line() \\
\quad \textbf{do if } line[1:3] \mathrel{==} \text{"##"} \\
\quad\quad \textbf{then if } line[3:6] \mathrel{==} \text{"■"} \\
\quad\quad\quad \textbf{then } { end_command(command, line[7:\texttt{*}line + 1]);\ break }
\end{array}
$$
```

`##include <file>.`

Includes file. Home directory for includes within included file is home directory of file relative to current home directory. I.e., if you include foo/bar (`.icn` is assumed), and foo/bar includes dot/zot, then we look for foo/dot/zot. If `-I` switch is present, don't bother doing includes.

Unable to render expression.

```
$$
\begin{array}{l}
do_include(arg) \Leftarrow \\
\quad \textbf{local } new_file \\
\quad cur_file \mathrel{:=} arg \\
\quad new_file \mathrel{:=} open(cur_file \ || \ \text{".icn"}, \text{"r"}) \\
\quad \textbf{if } /new_file \textbf{ then } write(\text{"ERROR: couldn't open "}, cur_file
\quad \textbf{else } { read_now \mathrel{:=} new_file \\
\quad\quad push(cur_files, read_now) \\
\quad\quad load_user_keywords(cur_file \ || \ \text{".keys"}) \\
\quad\quad process(\text{"include"}) \ # \text{ until ■ of file} \\
\quad\quad close(pop(cur_files)) \\
\quad\quad read_now \mathrel{:=} cur_files[1] } \ \blacksquare
\end{array}
$$
```

`##example` and `##end example`. Example paragraphs are left-justified and preceded by an appropriately numbered boldfaced "Example" keyword.

Unable to render expression.

```
$$
\begin{array}{l}
do_example(arg) \Leftarrow \\
\quad writes(\text{"\textbackslash fB Example.\textbackslash fR "}) \\
\quad process(\text{"example"}) \ \blacksquare
\end{array}
$$
```

`##code` and `##end code`.

Code is unjustified and Helveticized. Uncommented lines are processed as code. Commented lines bracketed by `##code` are treated similarly; the purpose is to present code examples in the file that are not to be seen by the ICON compiler.

Unable to render expression.

```
$$
\begin{array}{l}
do_code(arg) \Leftarrow \\
\quad \textbf{local } line \\
\quad write(\text{".nf0fH"}) \\
\quad \textbf{while } line \mathrel{:=} get_line() \\
\quad \textbf{do if } line[1:6] \mathrel{==} \text{"##■"} \textbf{ then break} \\
\quad\quad \textbf{else } pretty_print_line(line[2:\texttt{*}line + 1]) \\
\quad write(\text{".fi0fR"}) \ \blacksquare
\end{array}
$$
```

`##equations` and `##end equations`.

Typeset with TBL, one `.EQ` and `.EN.` per line, except that if the line is terminated by `\,` , continue equation on the next line.

Unable to render expression.

```
$$
\begin{array}{l}
do_equations(arg) \Leftarrow \\
\quad write(\text{".EQ"}) \\
\quad process(\text{"equations"}) \\
\quad write(\text{".EN"}) \ \blacksquare
\end{array}
$$
```

`##quote` and `##end quote`.

These are typeset with `.(q` and `.)q`.

Unable to render expression.

```
$$
\begin{array}{l}
do_quote(arg) \Leftarrow \\
\quad write(\text{".(q"}) \\
\quad process(\text{"quote"}) \\
\quad write(\text{".)q"}) \ \blacksquare
\end{array}
$$
```

`##table` and `##end table`.

Outputs `.TS` and `.TE` commands. Body is straight TBL.

Unable to render expression.

```
$$
\begin{array}{l}
do_table(args) \Leftarrow \\
\quad write(\text{".sp 4v0(c0TS"}) \\
\quad process(\text{"table"}) \\
\quad write(\text{".TE0)c0"}) \ \blacksquare
\end{array}
$$
```

For printing documentation lines. If the text following the `#` is white space, output a `.lp`

Unable to render expression.

```
$$
\begin{array}{l}
write_line(line) \Leftarrow \\
\quad \textbf{repeat if } upto(\text{' '}, line[1]) \textbf{ then } line \mathrel{:=} line[
\quad \textbf{if } \texttt{_}line = 0 \textbf{ then } write(\text{".lp"}) \textbf{ else } w
\end{array}
$$
```

For printing list lines.

Unable to render expression.

```
$$
\begin{array}{l}
plain_write_line(line) \Leftarrow \\
\quad \textbf{repeat if } upto(\text{' '}, line[1]) \textbf{ then } line \mathrel{:=} line[
\quad write(line) \ \blacksquare
\end{array}
$$
```

`pretty_print(line)` : For printing code. Output a `.nf` . Pretty print lines until end-of-file or comment. Output a `.fi` . Write-line, the comment if there was one.

Unable to render expression.

```
$$
\begin{array}{l}
pretty_print(l, command) \Leftarrow \\
\quad \textbf{local } line \\
\quad write(\text{".nf0fH "}) \\
\quad pretty_print_line(l) \\
\quad \textbf{while } line \mathrel{:=} get_line() \\
\quad \textbf{do if } line[1:2] \mathrel{==} \text{"#"} \\
\quad\quad \textbf{then } { write(\text{".fi0fR "}) \\
\quad\quad\quad\quad write(\text{".lp"});\ last_line \mathrel{:=} line;\ \bot } \\
\quad\quad \textbf{else } pretty_print_line(line) \\
\quad write(\text{".fi0fR "}) \ \blacksquare
\end{array}
$$
```

Pretty-print does special formatting in the following cases:

- Procedure definitions
- Control structures
- Reserved words
- User keywords

If the `-U<filename>` option is present, then keywords are read into the words table, with troff equivalents.

Unable to render expression.

```
$$
\begin{array}{l}
pretty_print_line(line) \Leftarrow \\
\quad \textbf{local } first,\ last,\ key,\ x,\ y \\
\quad { x \mathrel{:=} (upto((&amp;lcase \mathrel{+{+}} &amp;ucase \mathrel{+{+}} \text{'_0
\quad\quad \textbf{if } x = \texttt{_}line + 1 \textbf{ then } { writes(line);\ break } \\
\quad\quad y \mathrel{:=} (upto(\mathord{\sim}(&amp;lcase \mathrel{+{+}} &amp;ucase \mathre
\quad\quad key \mathrel{:=} (line[1:y] \ | \ \text{""}) \\
\quad\quad first \mathrel{:=} (line[1:x] \ | \ \text{""}) \\
\quad\quad line \mathrel{:=} (line[x:\texttt{_}line + 1] \ | \ \text{""}) \\
\quad\quad \textbf{while } \texttt{_}line &gt; 0 \textbf{ do } { \\
\quad\quad\quad \textbf{if } words[key] \mathrel{\sim}= \text{""} \textbf{ then } key \math
\quad\quad\quad last \mathrel{:=} line[y:\texttt{_}line + 1] \\
\quad\quad\quad writes(first, key) \\
\quad\quad\quad line \mathrel{:=} last } \\
\quad\quad write() } \ \blacksquare
\end{array}
$$
```

Unable to render expression.

```
$$
\begin{array}{l}
load_user_keywords(fname) \Leftarrow \\
\quad \textbf{local } w,\ a,\ x \\
\quad \textbf{if } not(w \mathrel{:=} open(fname, \text{'r'})) \textbf{ then } \bot \\
\quad \textbf{while } x \mathrel{:=} read(w) \\
\quad \textbf{do } { a \mathrel{:=} upto(\text{':'}, x) \\
\quad\quad words[x[1:a]] \mathrel{:=} x[a + 1:\texttt{*}x + 1] } \\
\quad close(w) \\
\quad \Uparrow \ \blacksquare
\end{array}
$$
```

Procedure definitions. Instead of the obvious

```
procedure F (a, b, c)
code
end
```

we use the logical-looking

Unable to render expression.

```
$$
F (a, b, c) \Leftarrow code \ \blacksquare
$$
```

Unable to render expression.

```
$$
\begin{array}{l}
\textbf{if } z \mathrel{==} \text{"■"} \textbf{ then } pretty_print_line(line \ || \ y \ ||
\textbf{else } { pretty_print_line(line) \\
\quad\quad \textbf{if } y \mathrel{==} \text{"■"} \textbf{ then } pretty_print_line(line \
\quad\quad \textbf{else } { z \mathrel{:=} get_line() \\
\quad\quad\quad\quad \textbf{local } y \\
\quad\quad\quad\quad y \mathrel{:=} get_line() \\
\quad\quad\quad\quad pretty_print_line(y);\ pretty_print_line(z) } }
\end{array}
$$
```

Control Structures: return, fail and every.

Unable to render expression.

x

Instead of `return x` we use *uparrow* , and for return we use ↑.
Instead of `fail` we use ⊥.

For `every i := 1 to j do C` we use `every i in 1, 2..j do C`

`every i := j to 1 by -1 do C` and `every x := !Y do C` we use `every i in j, j-1 .. 1 do C` and `every x in Y do C`

References

Balza84a. Stepehn R. Balzac, James H. Davenport, Patrizia Gianni, Richard D. Jenks, Victor S. Miller, Scott C. Morrison, Michael Rothstein, Christine J. Sundaresan, Robert S. Sutor, and Barry M. Trager, *Scratchpad II: An experimental computer algebra system*, Mathematical Sciences Department, IBM Thomas J. Watson Research Center, Yorktown Heights, NY 10598, May, 1984.

Dewar81a. Robert B.K. Dewar, Ed Schonberg, and Jacob T. Schwartz, *Higher level programming: Introduction to the use of the set-theoretic programming language SETL*, Courant Institute, N.Y.U., Summer, 1981.

Grisw83a. Ralph E. Griswold, "An overview of the Icon programming language (revised, September, 1985)," TR 83-3a, Dept. of Computer Science, University of Arizona, May, 1983.

Grisw83b. Ralph E. Griswold and Madge T. Griswold, *The Icon Programming Language*, Prentice-Hall, Inc., Englewood Cliffs, New Jersey, 1983.

Grisw85a. Ralph E. Griswold and William H. Mitchell, *Version 5.10 of Icon*, TR 85-15, Dept. of Computer Science, University of Arizona, August, 1985.

Ingal78a. D.H.H. Ingalls, "The SMALLTALK-76 programming system design and implementation," in *Fifth Annual ACM Symposium on Principles of Programming Languages*, pp. 9–16, 1978.

Knuth73a. Knuth, *The art of computer programming*, 1973.

Knuth82a. Knuth, Donald, "Web documentation system," *UNIX TeX Distribution Tape*, U. of Washington, 1982.

Kruch83a. Philippe Kruchten and Edmond Schonberg, *The Ada/Ed system: a large-scale experiment in software prototyping using SETL*, Computer Science Department, Courant Institute, New York University, 251 Mercer St., NY, NY, 10012, 1983.

Lampo83a. Lamport, Leslie, *The LaTeX Document Preparation System*, 1983.

Lipso81a. John D. Lipson, *Elements of Algebra and Algebraic Computing*, Benjamin/Cummings, 1981.

Loosa. Loos, Polynomial remainder sequences. *Computer Algebra* (ed. Buchberger).

NYU 84a. NYU Ada Project, *AdaSem: Static Semantics for Ada*, Ada Project, Courant Institute, New York University, 251 Mercer St., New York, NY, 10012, June, 1984.

Niven80a. Ivan Niven and H.S. Zuckerman, *An introduction to the theory of numbers*, 4th ed., John Wiley & Sons, 1980.

Yap85a. Yap, Chee, Polynomial remainder sequences and theory of subresultants. Unpublished lecture notes, NYU Courant Institute, Fall, 1985.

Yap86a. Chee Yap, Root Isolation, Unpublished lecture notes, N.Y.U., 1986.

Zippe86a. Richard E. Zippel, Algebraic Manipulation, Unpublished lecture notes, M.I.T., 1986.